Solution Path for Optimizing Monitoring and Observability

9 September 2024 - ID G00816740 - 68 min read

By: D.B. Cummings

Initiatives:IT Operations and Cloud Management for Technical Professionals; Evolve Service Management and Cloud Operations

To resolve incidents quickly, operations engineers rely on modern and mature solutions to collect and analyze telemetry. This research guides operations architects in enabling the monitoring of critical infrastructure and applications and implementing observability to resolve problems.

Overview

Key Findings

- Monitoring and observability strategies must employ the collect-ingest-analyze-act cycle to promote health and performance to support business objectives.
- Organizations benefit from complementing traditional monitoring with observability to measure business impacts, understand end-user experience and equip technology teams with performance data.
- AI and automation cannot replace skilled resources. Instead, they will support and augment resources by reducing research time, resolving problems and allowing resources to tackle more critical business challenges.
- Organizations face the twin challenges of adoption and perceived value for observability. Overcoming either will require skilled in-house practitioners.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Recommendations

- Improve the effectiveness of your monitoring and observability initiatives by focusing on the most critical services, user journeys and digital experiences.
- Adopt a “shift-left” strategy for telemetry collection by integrating instrumentation early in the development life cycle and creating a single telemetry data source for development and operations analysis.
- Develop efficient observability architectures by formalizing the strategy and ownership of telemetry data management, flow and filtering.
- Shorten the path to improved performance and availability by shifting from a unified analytic approach to a layering approach for analysis of monitoring data, observability telemetry and event streams.
- Promote continuous improvement of your visibility strategies, solutions and processes by building a collaborative community with motivated practitioners driven to share knowledge and understanding.

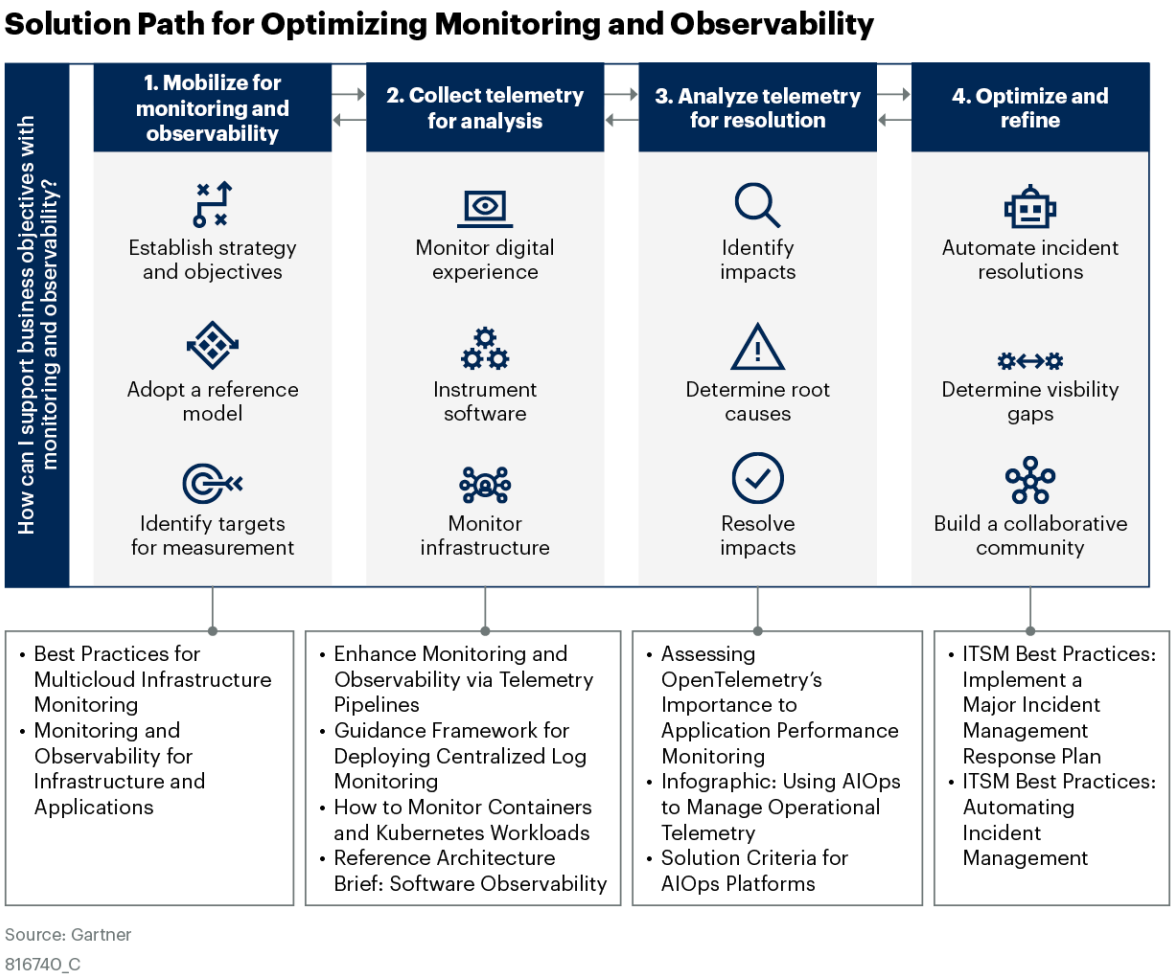
Problem Statement

Operations engineers require actionable insight from analyzing targeted application software and infrastructure telemetry data to ensure workloads consistently meet business objectives.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Solution Path Diagram

Figure 1: Solution Path for Optimizing Monitoring and Observability



Solution Path

Monitoring and observability strategy and technology evolve with application architectures. But most organizations find themselves still supporting mission-critical legacy applications and technologies. Organizations need a clear direction for optimizing their monitoring and observability strategy and solutions to effectively support today's needs while being ready to meet tomorrow's challenges.

This Solution Path provides direction for monitoring critical infrastructure and applications and implementing observability to support problem detection and resolution.

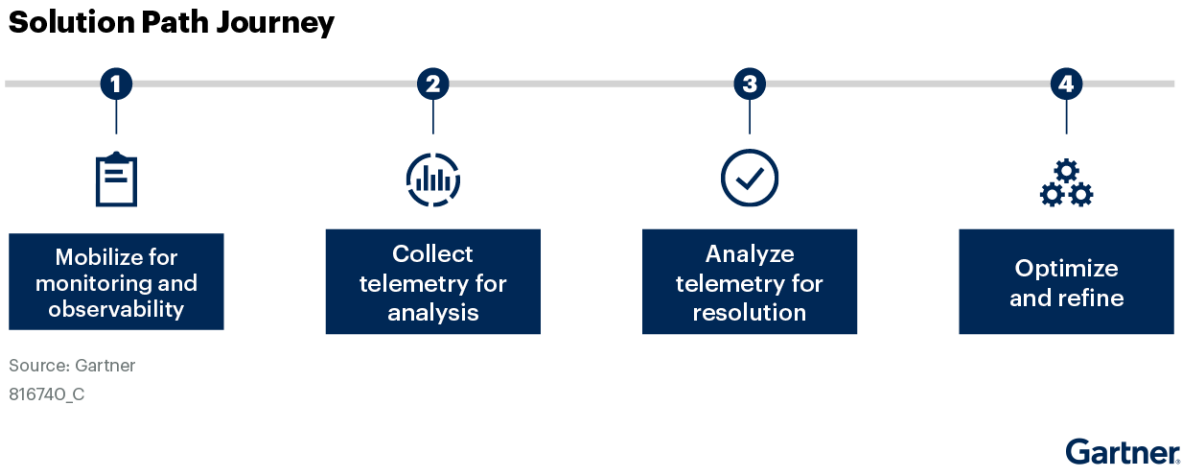
The steps in this Solution Path are based on the following assumptions:

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- You have existing technology investments in monitoring or observability tools.
- Your organization is responsible for supporting and maintaining applications and software services, whether internally developed or COTS.
- Your business-critical applications leverage cloud, cloud-native or legacy technologies and architectures.
- You need to accurately measure the impact of your vendors' SaaS and PaaS performance.
- You want to maximize the value of your current tools while developing a roadmap to address gaps in your monitoring and observability strategy and coverage.

The Solution Path for Optimizing Monitoring and Observability provides examples of ways to use telemetry data for insights into delivering better, more efficient services. The Solution Path journey comprises four steps (see Figure 2).

Figure 2: Solution Path Journey



The steps of the Solution Path are:

1. **Mobilize for monitoring and observability:** Plan and prepare to successfully implement and manage an enterprise monitoring and observability practice.
2. **Collect telemetry for analysis:** Deploy mechanisms, solutions and processes to collect telemetry data.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

3. **Analyze telemetry for resolution:** Prevent or resolve business-impacting problems by leveraging insight from the analysis of telemetry data.

4. **Optimize and refine:** Continuously improve the business value gained from monitoring and observability through automated problem resolution, gap analysis and effective adoption of best practices.

Although the steps in the Solution Path are arranged chronologically, organizations will find that working through them is dynamic when considering the strategy, tools and data lanes. The Solution Path diagram shows that feedback informs and improves each prescribed activity. In addition, new services and applications will be more easily added incrementally once the prescribed solutions and processes are in place.

The Solution Path references Gartner's published research on related topics.

Table 1 includes a guide to critical terminology used in this research. Other terminology is defined in-line.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 1: Terminology

Term	Definition
Instrumentation	Code or tooling that captures data about the state of a monitored entity
MTTD	Mean time to detect
MTTR	Mean time to restore
SLI	Service-level indicator: A quantitative measure of some aspect of the level of service being provided
SLO	Service-level objective: A target value or range of values for a service level that an SLI measures
SRE	Site reliability engineering
Telemetry	Data generated by a monitored entity that documents its state

Source: Gartner

Step 1: Mobilize for Monitoring and Observability

Building a solid foundation is vital to ensure a lasting structure. In Step 1, you will lay a solid foundation for the insight needed to solve your organization's application and infrastructure health and performance problems. Building a proper foundation requires understanding the strengths and weaknesses of your materials. Although monitoring and observability are essential building materials, their definitions and relationship to one another are frequently misunderstood. These definitions of monitoring and observability are used throughout this research.

> **Gartner's Definitions of Monitoring and Observability**

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

> **Monitoring:** Monitoring is an active process that involves collecting telemetry data to identify when something is negatively impacted and supporting the notification of support staff or triggering automation to take corrective action.
>
> **Observability:** Observability is a property rather than a process. Observability exists when suitable telemetry is available regarding the internal state of a system to identify the cause of an anomaly or malfunction even when such a malfunction has never happened before. Tools alone cannot provide observability, but they can facilitate it.

Observability can't be delivered by a tool alone. Observability is in the DNA of the software. If you don't build an observable application, no tool will make it observable.

This research will refer to both monitoring and observability solutions (or tools) as "telemetry analysis solutions" because telemetry data is sent to these solutions for analysis. This may lead to a misunderstanding that a telemetry analysis solution alone can meet the goals of insight and understanding into business-impacting problems. This is not the case. Telemetry analysis solutions require targeted telemetry to generate accurate understanding and insight.

Although AIOps is referenced throughout this research, the term AIOps will not refer to stand-alone solutions, tools or platforms. Instead, it will describe the general concepts of advanced analytics, machine learning (ML) and other AI features applied to telemetry data. Another term, event intelligence solutions, will refer to solutions that analyze events and other telemetry from multiple sources.

Few organizations start from scratch. Every organization has some kind of monitoring in place. In the last few years, many organizations have been revisiting their monitoring strategies and capabilities to determine how best to support observability.

The goal of observability must be to move away from the event-driven reactivity of the past toward a data-driven model that will meet today's challenges and prepare for tomorrow's unknowns. This move does not mean shifting away from monitoring toward observability. Instead, monitoring and observability work together to provide insight into detecting and solving problems.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

For Step 1: Mobilize for monitoring and observability, you must:

- **Establish your strategy and objectives:** Before you build a house, you need to lay a foundation. The same is true for monitoring and observability. You must define the problems you are trying to solve and the approach for solving them. The problems are your objectives, and the approach is your strategy.
- **Adopt a reference model for monitoring and observability:** To build a house, you need a blueprint. The monitoring and observability reference model is the blueprint for supporting the telemetry life cycle and clearly defines the categories and terminology. It will enable you to map and assess your existing tools and solutions to determine whether to replace or augment your portfolio. In turn, you can develop a roadmap for adopting and integrating solutions to understand the health and performance of your business services.
- **Identify your targets for measurements:** A house requires appliances and furnishings to be functional. Service-level objectives (SLOs) furnish monitoring and observability data with a method to measure business impacts. You will use SLOs to prioritize the impact and urgency of incident response and serve as a proxy for quantifying your users' happiness.

Step 1.1: Establish Strategy and Objectives

As with any process improvement or transformation-related initiative, understanding the scope and building a strategy for moving to the desired state is the first step. You might be asking a few questions:

- What are the business objectives we are trying to support?
- Where are the pain points?
- Are the users of our business services happy with their performance and availability?
- Are developments or best practices not followed because we're unaware of them?

Common Challenges

The following examples indicate monitoring and observability challenges for which Gartner clients regularly seek help:

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **Agility:** As the pace of change accelerates, longstanding processes and tools often no longer provide the value they once did. Automation can sometimes assist here, as can adopting additional DevOps practices such as better feedback loops and faster iteration. You must explicitly adapt monitoring and observability properties to accommodate agility.

- **Modernization:** Traditional monitoring tools and practices may not support cloud providers and frameworks such as public cloud, containers and Kubernetes. In addition, support workflow and diagnostic processes often need to evolve.

- **Visibility:** Digital experience data, in particular, is not typically available within traditional infrastructure monitoring solutions, which generally monitor from the perspective of resource availability and health at the device level. A good digital experience depends on sufficient resources and healthy infrastructure, but the relationship is indirect. This indirect relationship makes it impossible to associate application performance directly with infrastructure.

- **Coordination:** Traditional monitoring practices organized around technology rather than services may lead to inconsistent and siloed alerting that causes specialist teams to investigate the same problem from different angles. Instead, an agreed-on, single source of telemetry data speeds resolution by reducing rework and enabling specialist teams to collaborate.

- **Diversity:** Telemetry and events from disparate sources (e.g., monitoring tools) need to be integrated into a telemetry analysis solution or event intelligence solution. Additionally, acquisitions and mergers can further complicate this problem by introducing redundancies into the system.

- **Organizational:** Many infrastructure and service delivery organizations need help to overcome the meantime-to-innocence problem as a working paradigm. The insight gained from monitoring and observability data is an antidote to this problem.

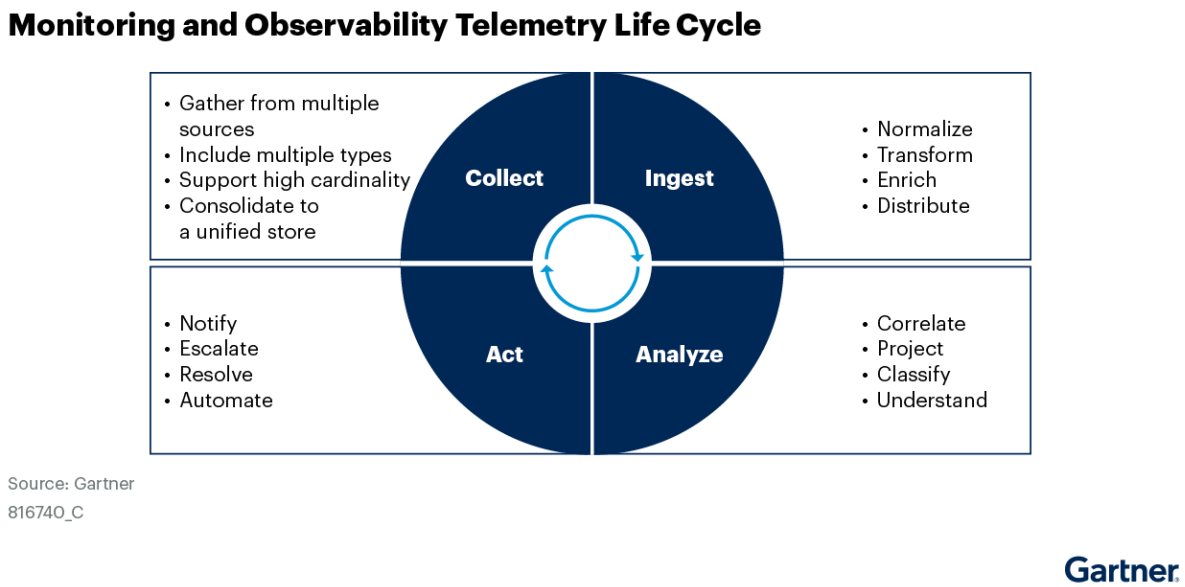
Undoubtedly, these challenges will resonate with your experiences and bring other challenges to mind.

The Monitoring and Observability Telemetry Life Cycle

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

To address your questions and challenges, you will need to adopt the Monitoring and Observability Telemetry Life Cycle as your foundation. Figure 3 illustrates this life cycle.

Figure 3: Monitoring and Observability Telemetry Life Cycle



Some readers may recognize Figure 3 as a variation of John Boyd's OODA Loop, [1] but its application here is less tactical. Instead, it serves as a way to manage input from multiple sources systematically, interpret the data and decide about the next step as quickly as necessary. The importance of this life cycle will become apparent throughout this research.

Strategies and Objectives

Beginning with Step 1.2, you will understand how the life cycle is defined and practically implemented. The life cycle will become the strategy by which you achieve your objectives. Those objectives will be to predict and prevent or detect and resolve business-impacting problems. Step 1.3 will introduce the measurement of business objectives with SLOs. Step 3 covers how user journeys can leverage the SLIs captured in the telemetry life cycle to accurately measure your SLOs.

Aligning With Larger Transformation Initiatives

With the focus of product development on value stream management and DevOps practices continuing to become commonplace, the traditional I&O organization structured around silos has become an impediment. Optimizing monitoring and observability requires adopting a "platform" perspective. This perspective dovetails nicely with DevOps and SRE goals and practices.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Position I&O as a facilitator. Provide access to the tools, techniques and instrumentation that enable product teams to ensure services meet their SLOs and perform the necessary diagnostic and repair activity when they do not.

For many organizations, the overarching concerns represent a combination of these strategic issues:

- Existing monitoring solutions that no longer meet the organization's needs
- Health and performance data spread across disparate tools and teams
- Emphasis on monitoring technology and facilities rather than applications and services
- Increasing adoption of cloud services and SaaS applications for critical business workloads
- Failure of teams to adopt deployed monitoring or observability solutions

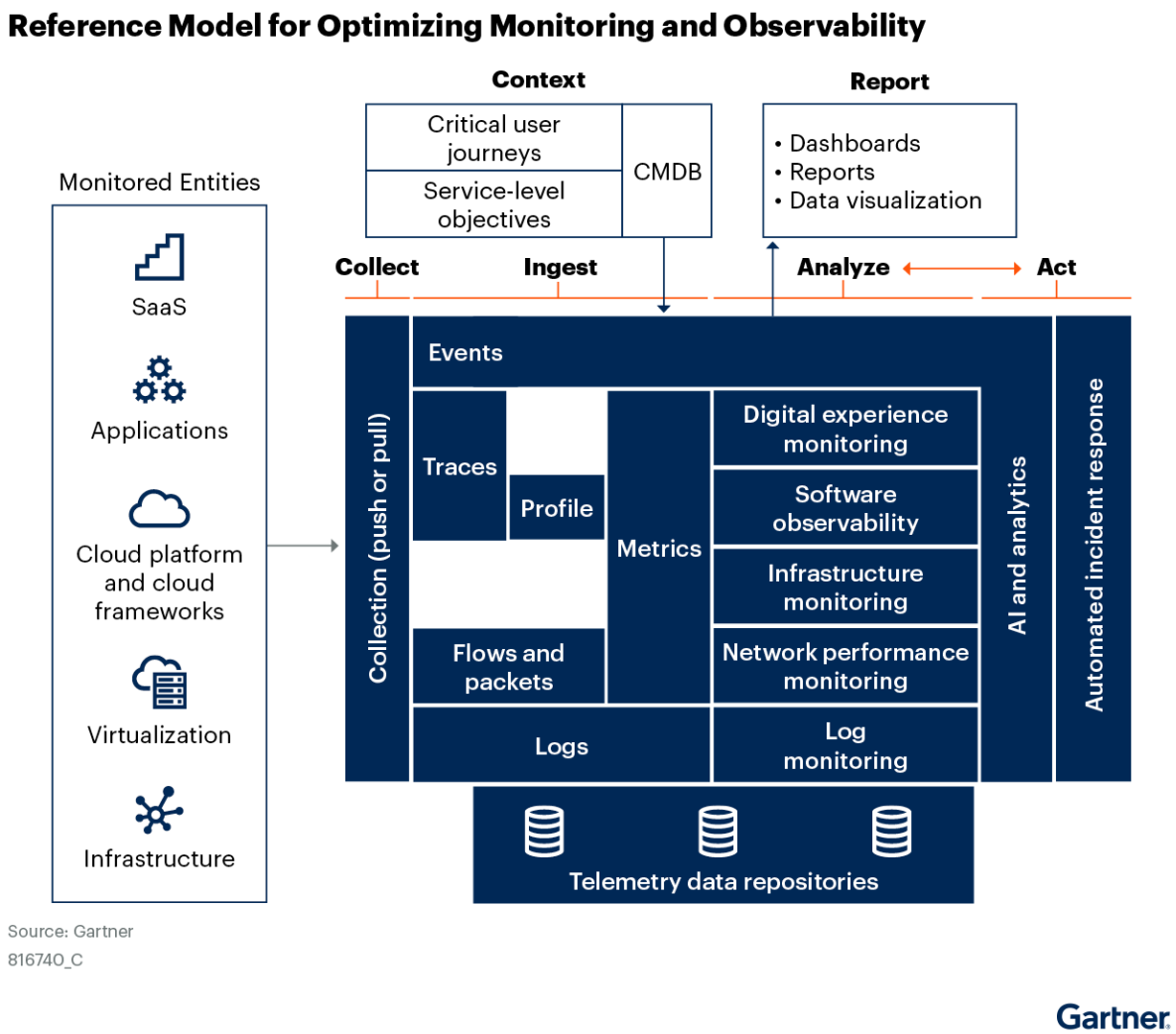
Step 1.2: Adopt a Reference Model

Traditionally, investments in monitoring solutions focused on infrastructure. As workloads moved to cloud providers or highly abstracted environments, organizations recognized a need to shift to understanding the health and performance of business-critical services. Understanding software performance has always been a need for I&O, but the reality of that need is more apparent as cloud applications become more abstracted from the infrastructure layer.

Traditional monitoring is still vital in identifying changes in the health and performance of critical systems and services. As a complementary discipline, observability provides insight into identifying and solving existing problems or predicting future ones. Figure 4 illustrates a reference model for optimizing monitoring and observability.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 4: Reference Model for Optimizing Monitoring and Observability



As you initially work through this Solution Path, you will not have deployed everything represented here. But, ultimately, your organization should have a model like this.

A Tour of the Reference Model

As you read the description of the reference model, keep in mind the terminology used in Figure 3 for the phases of the monitoring cycle. As defined in Table 1, this research uses the term “telemetry” to refer to health and performance data being collected and examined for insight and understanding and the term “instrumentation” to refer to capturing the telemetry data.

Collect

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Telemetry travels from instrumented entities to telemetry analysis solutions in various ways. Different software components or frameworks running on a single computer instance, virtual machine (VM), Kubernetes cluster or container may be instrumented by multiple collection methods:

- **Agent:** This software component gathers telemetry data about the entity on which it is running and makes it available to a telemetry analysis solution. Agent software may be provided by your monitoring or observability solution or built into an operating system (e.g., Simple Network Management Protocol [SNMP]).
- **Collector:** This software component gathers multiple types of telemetry data (e.g., traces, metrics, logs) from multiple instrumented entities and makes the data available to a telemetry analysis solution. Collectors are used with SaaS telemetry analysis solutions to simplify telemetry transmission from a data center or cloud provider to the SaaS telemetry analysis solution.
- **Push:** Telemetry travels from an instrumented entity to a telemetry analysis solution without being explicitly requested.
- **Pull:** A telemetry analysis solution requests telemetry from instrumented entities.

Distinguishing between agents and other software mechanisms (e.g., exporters, forwarders) for gathering telemetry data can be challenging as capabilities overlap, making precise definitions difficult. See Reference Architecture Brief: Software Observability for more details on the software mechanisms for gathering and collecting telemetry. The sections on Code-Level Instrumentation and Infrastructure Visibility expand on the types and examples of mechanisms used for instrumentation. The Telemetry Collectors section clarifies the collectors' role and function.

In network monitoring, where SNMP has long been quite common, the customary five-minute granularity for counter aggregation is no longer sufficient. Instead, publish-subscribe-based streaming telemetry allows a monitoring solution to request that telemetry stream transmit from the monitored entity to the monitoring solution on a continuous basis. Beyond this description, streaming telemetry mechanisms are not discussed in this research.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

> **The instrumentation capture and telemetry transmission processes should be designed intentionally to minimize the impact on monitored entities.**

Ingest

The scope of the ingest phase is substantial. The most common types of telemetry are:

- **Metrics (or time series data):** A metric is a numeric value and a time stamp accompanied by metadata.
- **Logs:** A log is a text string or structured data, a time stamp and metadata.
- **Traces:** A trace is a specialized type of telemetry that encapsulates a single operation within an application.
- **Events:** Events have a format similar to log data but are typically in a structured format.

The term "event" can be overloaded. In monitoring and observability, an event is intentionally generated by a software or system and intended to describe a change in state. Some of these changes in state might be merely informational (e.g., backup is complete). But, typically, they represent a change in state that is "interesting," which means it is indicative of a negative impact on an application or software service.

Ingest may also include the telemetry storage mechanism. An essential aspect of today's telemetry analysis solution is analyzing and correlating monitoring data from multiple sources. This analysis does not necessarily require a single repository, but such options are available. See the Telemetry Data Store section in Reference Architecture Brief: Software Observability for more information.

Analyze

In the analyze phase, domain-specific tools translate the telemetry into the dashboards, reports and indicators the organization requires. Context — nontelemetry data — is usually needed for analysis to be effective. Three types of common context are:

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **Static context:** This is context in the form of SLOs, metadata from a configuration management database (CMDB) or critical user journey data that defines the highest-priority transactions to monitor.

- **Dynamic context:** This is context collected by an action responding to an event. An example would be a log of recent changes to an environment.

- **Topological context:** This context defines the components used to deliver a service. It is most commonly used in event analysis and combines durable and dynamic information.

Event management determines the subsequent actions for events generated from the data analysis of telemetry analysis solutions. This may include creating alerts for incidents (e.g., ticket creation), notifications or triggering automation to collect additional information or resolve the event.

Incorporate event management into your monitoring and observability ecosystem to extend insight beyond telemetry analysis solutions by applying analysis to events generated from multiple sources. For more guidance, see Infographic: Using AIOps to Manage Operational Telemetry.

The way to incorporate event management is through the use of AIOps in the following ways:

- **Data and analytics tools:** Solutions that allow operators and data scientists to create targeted and customized analyses.

- **Observability solutions:** These telemetry analysis solutions leverage ML, AI or advanced analytics on telemetry data.

- **Event intelligence solutions:** These solutions ingest events and potentially other telemetry data sent from other telemetry analysis solutions to detect patterns and find root causes to augment, accelerate and automate resolution.

Act

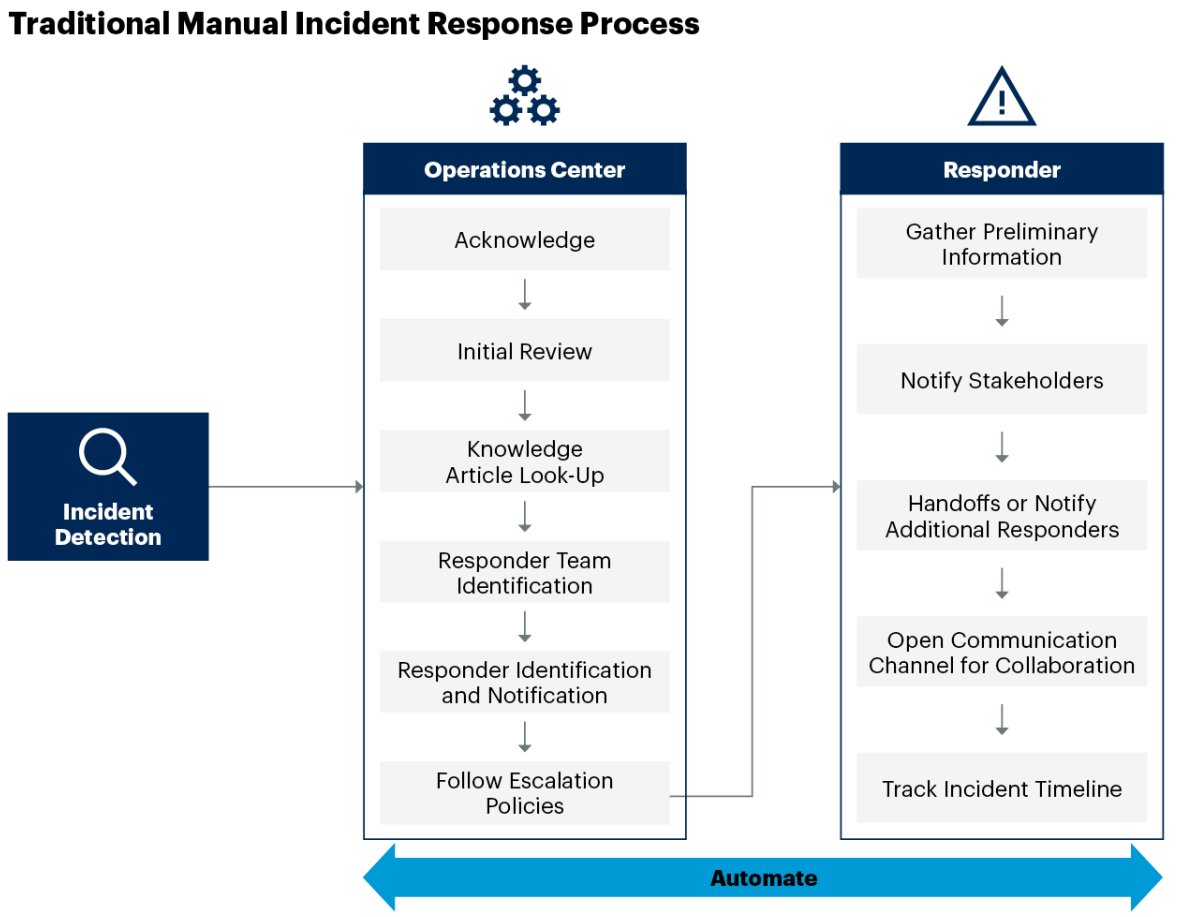
The analyze and act phases often work cyclically, where the action may result in additional events and more actions. The traditional workflow still exists in which a ticket is opened and sent by email or text for an anomalous signal in an analysis domain. However, you must coordinate many more aspects of this workflow.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Gartner defines automated incident response (AIR) as a solution that automates incident response by enabling centralized alert or incident routing with a policy- or rule-based engine, on-call scheduler or streamlined collaboration that can improve operational efficiencies with actionable insights. For more information on automating incident resolution, see ITSM Best Practices: Automating Incident Management.

Figure 5 illustrates a typical manual incident response workflow in an organization. Operations centers can consume significant resources if there is a manual process for every alert generated by telemetry analysis solutions. The traditional incident response approach presents considerable challenges, including significantly reducing manual processes and the need for more collaboration between teams to conduct triage.

Figure 5: Traditional Manual Incident Response Process



Source: Gartner
728681_C

Gartner

Tool Rationalization

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Although hybrid strategies exist, and pursuing a clean slate is rare, your tool philosophy can make or break the solution. Many organizations discover overlap among existing solutions. When two or more solutions each meet 80% to 90% of your requirements in an existing domain, you should consolidate them.

Step 1.3 Identify Targets for Measurement

Many I&O organizations prioritize infrastructure monitoring under the often erroneous assumption that if the infrastructure appears healthy, the associated application is healthy, and the application's users are happy. There is also a tendency to adopt entirely separate monitoring solutions to cover different types of infrastructure. These disparate solutions encourage a problem resolution workflow often described as "mean time to innocence," where each potentially responsible team works to prove the situation has not originated within its areas of responsibility.

Forms of Telemetry

Regardless of the source, the telemetry you want will rarely be in a form other than metrics, logs, events or trace data. Trace data is typically generated by instrumented applications or an application layer component like a service mesh, while metric and log data may be generated by almost anything.

Monitoring ensures systems operate as expected, and metrics are usually the key. Trace data with associated metrics and logs support observability in analysis and diagnostics for insight and understanding.

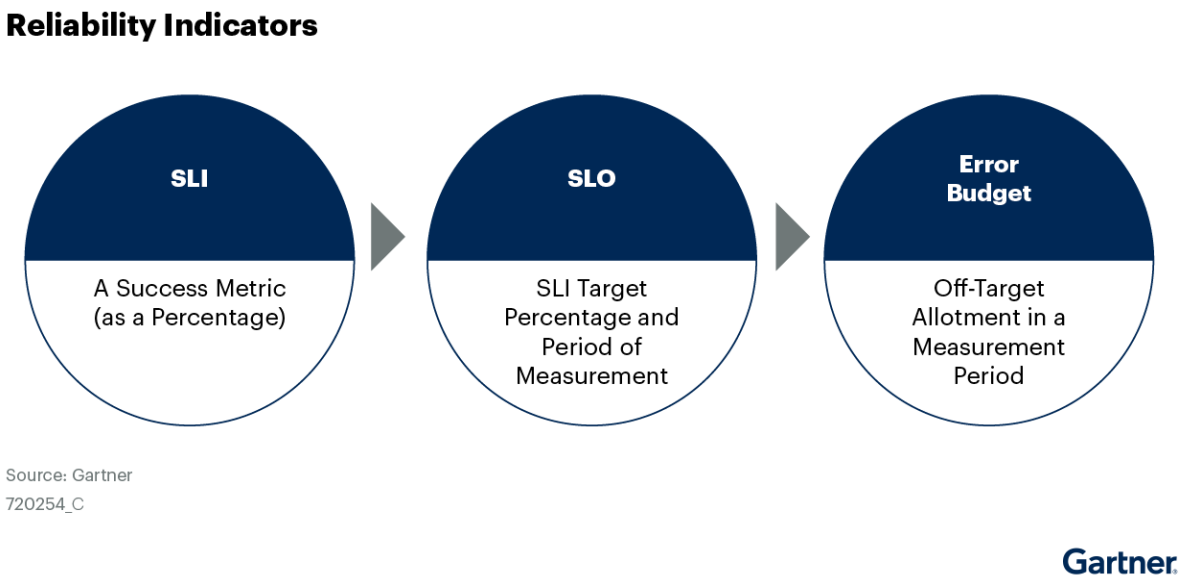
The Language of Monitoring and Observability

The only way to establish that anything is operating "as expected" is to have expectations. Those expectations must be reasonable, achievable and testable. Heavy reliance on SLOs is critical to optimizing monitoring and observability. A well-defined SLO is directly associated with an indicator important to a system's users or the product owner as a proxy for the users. These objectives encompass availability, performance, resilience, feature release cadence and other aspects. SLOs and error budgets provide a measurable, realistic alternative to the often unstated assumption that every system is available 100% of the time.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

SLOs, SLIs and error budgets have become pervasive in site reliability engineering (SRE) vocabulary. But organizations do not have to fully adopt SRE to benefit from SLOs' value-targeted approach. Figure 6 illustrates the relationship between these reliability indicator measurements.

Figure 6: Reliability Indicators



Choosing representative and meaningful SLIs is critical. In most cases, an infrastructure-based metric (e.g., available memory, percentage of free worker nodes) is unlikely to be meaningful because it is not often something that service users care about. Choose SLIs that directly measure the users' experience with your service. Your site reliability engineers may care about the percentage of free worker nodes when troubleshooting a latency problem, but that is a different use case.

Table 2 includes two examples of better SLIs.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

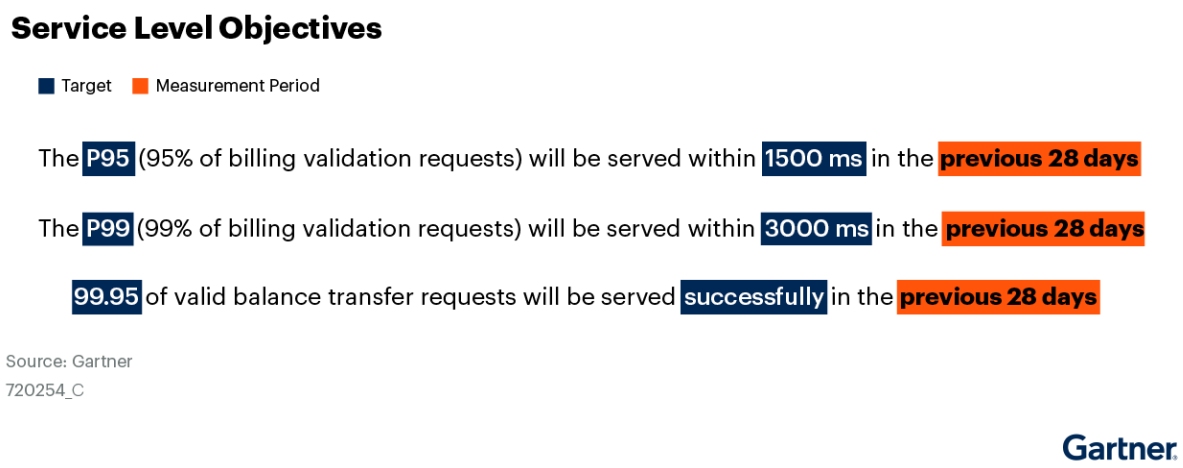
Table 2: SLI Examples

SLI	Type
The proportion of billing validation requests served in “X” milliseconds or less, measured at the API gateway	Latency
The proportion of valid balance transfer requests served successfully	Availability

Source: Gartner

An identified SLI can be used in one or more SLOs. Both the target percentile and the measurement period can vary. Figure 7 includes examples of SLOs based on the SLIs shown in Table 2.

Figure 7: Service-Level Objectives



The error budget represents the difference between “perfect” and the SLO. For the SLOs in Figure 7, the error budget is 5%, 1% and 0.05%, respectively, during the same measurement period.

SLOs measure symptoms, not causes. Adopting the SLO, SLI and error budget mechanisms facilitates a shift in perspective from reactive to proactive, preparing for even more advanced analytics. As you establish SLOs, you may be asked to translate availability percentages into time in your conversations with product ownership. Table 3 provides a reference for doing so.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 3: Availability Objectives Translated to Time

(Enlarged table in Appendix)

Availability objective	Outage allowed per year	Outage allowed per quarter	Outage allowed per month	Outage allowed per week	Outage allowed per day	Outage allowed per hour
90%	36.5 days	9 days	3 days	16.8 hours	2.4 hours	6 minutes
95%	18.25 days	4.5 days	1.5 days	8.4 hours	1.2 hours	3 minutes
99%	3.65 days	21.6 hours	7.2 hours	1.68 hours	14.4 minutes	36 seconds
99.5%	1.83 days	10.8 hours	3.6 hours	50.4 minutes	7.2 minutes	18 seconds
99.9%	8.76 hours	2.16 hours	43.2 minutes	10.1 minutes	1.44 minutes	3.6 seconds
99.95%	4.38 hours	1.08 hours	21.6 minutes	5.04 minutes	43.2 seconds	1.8 seconds
99.99%	52.6 minutes	12.96 minutes	4.32 minutes	60.5 seconds	8.64 seconds	9.36 seconds
99.999%	5.26 minutes	1.3 minutes	25.9 seconds	6.05 seconds	0.87 seconds	0.04 seconds

Source: Gartner

If more relevant to your system, you can construct a similar table using request volume or transactions rather than time. For example, assuming a constant request rate of 100 transactions per second, which is equivalent to 8,640,000 transactions per day, and an SLO of 99%, the error budget is 86,400 per day or 604,800 per seven days. This is equivalent to 1.68 hours of 100% failure per week.

For some services, request rate may be a more effective way to represent degradation or partial failure than time.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Actionable SLOs should not be aspirational but should be based on past performance and negotiated based on business needs. They must also be realistic, achievable and available to report on. Recall that they will ultimately be associated with entire user journeys, not necessarily discrete services. There is a school of thought that allows for aspirational SLOs as a way to visualize progress toward a currently achievable goal. Such SLOs would not trigger an alert or any action but are identified and available in reports or dashboards for periodic assessment. Pursue aspirational SLOs only after your organization has attained familiarity with this process.

Burn Rate

As Table 3 suggests, even at 99% ("two nines") over 28 days, there is not much time available for 100% outages if you wish to remain compliant with your SLO. Regardless of the specific SLO, moderate the response to an available event by measuring the burn rate. Some general rules for alerting based on error budget consumption rate include:

- **Fast burn:** If the current error rate will exhaust the error budget within 24 hours, alert for immediate action — such as an immediate alert to the support team.
- **Slow burn:** If the current error rate (as a constant) will exhaust the error budget before the reporting period ends, alert for nonemergency action — such as opening a ticket.

Additional Guidance

Individual user-facing services make for a straightforward illustration. In a steady state, however, these SLOs are attached to composite user journeys that comprise multiple requests through multiple services to accomplish a business objective. To attach an SLI to a user journey, you will need SLIs for each service that contributes to the journey. Read about critical user journeys in Step 3: Analyze Telemetry for Resolution.

Favored techniques for the monitoring and observability of service-based applications emphasize the use of RED signals, which is an acronym for:

- **Rate:** Request rate, usually per second
- **Errors:** Errors and failed requests, usually per second
- **Duration:** Request duration, usually per second

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

These signals can often provide the raw data for SLIs for many services and are an excellent place to start. For more details, see Monitoring and Observability for Infrastructure and Applications.

Many telemetry analysis solutions (i.e., monitoring and observability) now include native SLO monitoring and reporting support. These solutions can significantly simplify the building and reporting aspects, but obtaining value from them requires the same careful and consensus-driven approach to establishing meaningful, actionable SLIs and SLOs. The following are examples of telemetry analysis solutions that support SLO reporting:

- AWS CloudWatch
- Coroot
- Datadog
- Dynatrace
- Elastic
- Google Cloud Observability
- Grafana
- Honeycomb
- New Relic

Enforcing SLOs requires business or product owner buy-in, and failure to obtain organizational consensus on either the error budget or SLOs is the most common reason these initiatives fall apart. On the other hand, building a culture around SLOs is one of the best ways to position yourself for success.

Read More

Best Practices for Multicloud Infrastructure Monitoring

Improve Customer Experience and Reliability With SLIs and SLOs

Infographic: Using AIOps to Manage Operational Telemetry

Monitoring and Observability for Infrastructure and Applications

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Step 2: Collect Telemetry for Analysis

Step 2 in optimizing monitoring and observability ensures the initial structures are in place to collect, store and analyze telemetry.

One crucial aspect intentionally left open-ended is whether to create a separate monitoring (or observability) framework for SLO-related signals or apply these methods to an organization's existing practices and tools. Logically, as these concepts are illustrated, it is convenient to frame them in isolation. In a perfect world, we should position SLO monitoring as a tier above whatever monitoring (and observability) occurs.

> **Implementing SLOs is a good choice, as it provides evidence of the value of the existing solutions and prioritizes replacement opportunities that offer more accurate impact analysis and response.**

Organizations will likely approach this step differently; many will not get it right the first time. Operations architects may instinctively grasp the value of SLOs but then need to lobby for internal support or sponsorship. Ironically, the chaos and instability an organization is experiencing may directly correlate to its openness to rethinking performance and availability monitoring.

Digital experience monitoring (DEM) and software observability are the primary telemetry sources for defining SLIs and measuring SLOs associated with availability, latency, throughput or transaction failure rate. The rest of this step centers on collecting and storing additional telemetry from infrastructure and applications. The collection mechanisms, such as agents and collectors, and where and how to store telemetry are discussed. Given these solutions' popularity, capabilities and integration with cloud-based technologies, the recommendations favor SaaS or hybrid architectures.

Step 2.1: Monitor Digital Experience

Gartner defines DEM as a performance analysis category that provides visibility into the end users' experience as they interact with applications, services and related resources. Due to its direct relationship with the application, DEM provides an excellent basis for SLIs.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 8 illustrates the DEM monitoring scope compared with traditional monitoring. Infrastructure monitoring provides visibility into only data center and cloud infrastructure. DEM delivers visibility into your customer problems related to the device, browser, regional networks, internet routing, Domain Name System (DNS) and content delivery network (CDN). DEM can proactively identify issues because client-side customer problems are not visible through server-side metrics, logs and traces.

Figure 8: DEM in the Monitoring Universe

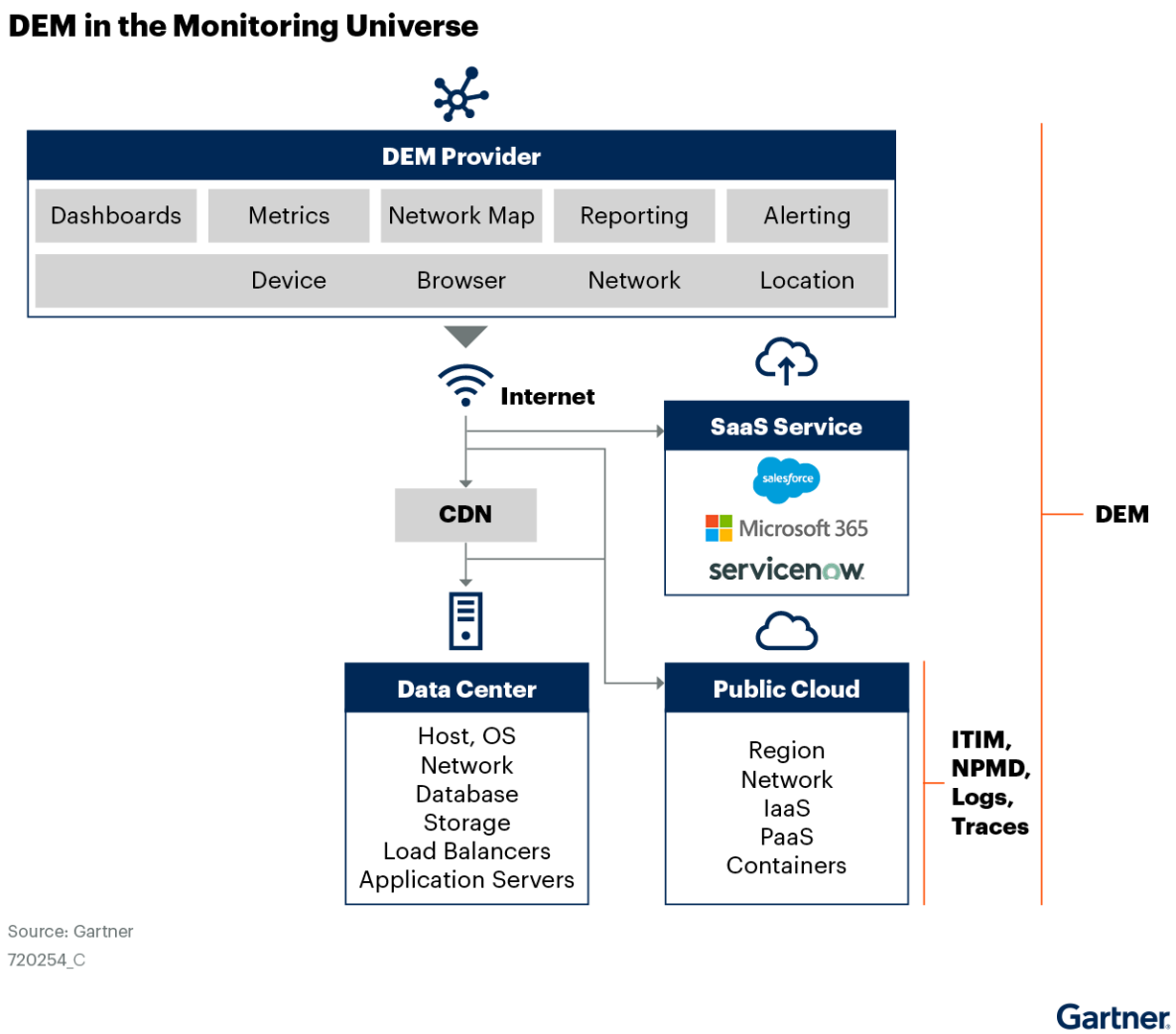


Table 4, reproduced from the Market Guide for Digital Experience Monitoring, illustrates some common I&O use cases for DEM technologies.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 4: DEM Technologies and Most Common I&O Use Cases

(Enlarged table in Appendix)

DEM technology	Use cases
Real user monitoring	▪ Monitoring performance and quality of user experience for external-facing applications (browser and native mobile app) ▪ SLA compliance ▪ Root cause analysis of front-end application performance problems ▪ User experience analysis through session replay ▪ Omnichannel user journey and customer experience analysis ▪ Business outcome analysis
Synthetic transaction monitoring	▪ Monitoring SaaS applications by simulating multiple step transactions and monitoring the performance at each step ▪ Identifying problems across endpoint, local network, internet and application servers ▪ Testing from remote locations such as last-mile, cloud providers or Tier 1 providers ▪ Benchmarking website availability and performance ▪ API monitoring ▪ Monitoring wireless access points ▪ Monitoring network and internet user traffic ▪ Testing performance in preproduction
Market Guide for Digital Experience Monitoring	

Source: Gartner

How to Monitor Digital Experience

There are two DEM technologies:

- **Real user monitoring (RUM):** RUM measures user interactions, typically through browser instrumentation.
- **Synthetic transaction monitoring (STM):** STM simulates or emulates user interactions via an agent.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 5 contains examples of DEM vendors listed alphabetically along with links to the webpages for their RUM or STM technology solutions.

Table 5: Example DEM Vendors With RUM or STM Solutions

(Enlarged table in Appendix)

Vendor	Real user monitoring	Synthetic transaction monitoring
Apica	NA	Apica Synthetic Monitoring
Broadcom	DX App Experience Analytics	DX App Synthetic Monitor
Catchpoint	Real User Monitoring (RUM)	Synthetic Monitoring
Cisco (AppDynamics)	Browser Real User Monitoring (Browser RUM) Mobile Real User Monitoring (Mobile RUM)	Browser Synthetic Monitoring
Cisco (ThousandEyes)	End User Monitoring	Browser Synthetics
ControlUp	NA	ControlUp Scoutbees
Datadog	Real User Monitoring (RUM)	Synthetic Testing and Monitoring
Dynatrace	Real User Monitoring (RUM)	Synthetic Monitoring
eG Innovations	Real User Monitoring	Synthetic Monitoring
Fortinet	NA	FortiMonitor
ITRS Group	ITRS Uptrends	ITRS Uptrends
Kentik	NA	Synthetic Testing and Monitoring
ManageEngine	Real User Monitoring (RUM), Site24x7	Site24x7
Netskope	Proactive Digital Experience Management (P-DEM)	NA
New Relic	Browser Monitoring	Synthetic Monitoring
Palo Alto Networks	Autonomous DEM	Autonomous DEM
Riverbed	Aternity Digital Experience Management Platform	Aternity Digital Experience Management Platform
SolarWinds	Pingdom	Pingdom Web Performance Monitor
Splunk	Splunk Real User Monitoring (RUM)	Splunk Synthetic Monitoring
2 Steps	NA	Synthetic Monitoring
Zscaler	NA	Zscaler Digital Experience (ZDX)
NA = Not available		

Source: Gartner

Out of the box, many of these solutions will be able to provide excellent insight into transactions through specific applications and will be able to inform application SLIs directly. Once you have monitored key transactions, defined SLIs and reported on SLOs, you can assemble these discrete transactions into user journeys. Then, you can attach SLOs to those user journeys and monitor them. Monitoring user journeys is covered in more detail in Step 3: Analyze Telemetry for Resolution. Organizations that have little experience with DEM should focus first on application-layer SLOs before defining SLOs for multiapp journeys.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Why Monitor Digital Experience?

Among the most common concerns Gartner clients express about their existing monitoring capabilities is being notified of problems by their customers, or users of their services, before they are aware of the problem. Because DEM captures the user experience directly, it is the ideal perspective to capture that missing signal. In addition, many of the SLIs necessary to monitor SLOs are related to or derived from DEM signals. Due to its reliance on SLOs, this research assumes that monitoring and observability include the digital experience. DEM does not end here, but the benefits and objects that underpin this Solution Path depend upon it.

Step 2.2: Instrument Software

If optimizing monitoring and observability requires nothing more than agreeing on an error budget and establishing SLOs informed by SLIs that meaningfully correspond to actual digital experience, why can't the Solution Path stop here?

In theory, the Solution Path could stop here — unless your organization is also responsible for restoring systems to service after they break or a degradation threatens to consume the entire error budget for the month in 30 minutes. In those cases, you must answer two questions: How do you know what to fix? What other symptoms exist beyond the SLI, indicating that user experience is threatened? To diagnose and resolve impacting deviations, you must gather the correct telemetry data about your software and services.

Understanding performance deviations requires software instrumentation and visibility into the underlying infrastructure, whether you need insight into traditional or cloud-native applications. This section covers the instrumentation needed to help address challenges like identifying where a user is experiencing poor application performance in a complex environment: code-level instrumentation and kernel instrumentation.

Code-Level Instrumentation

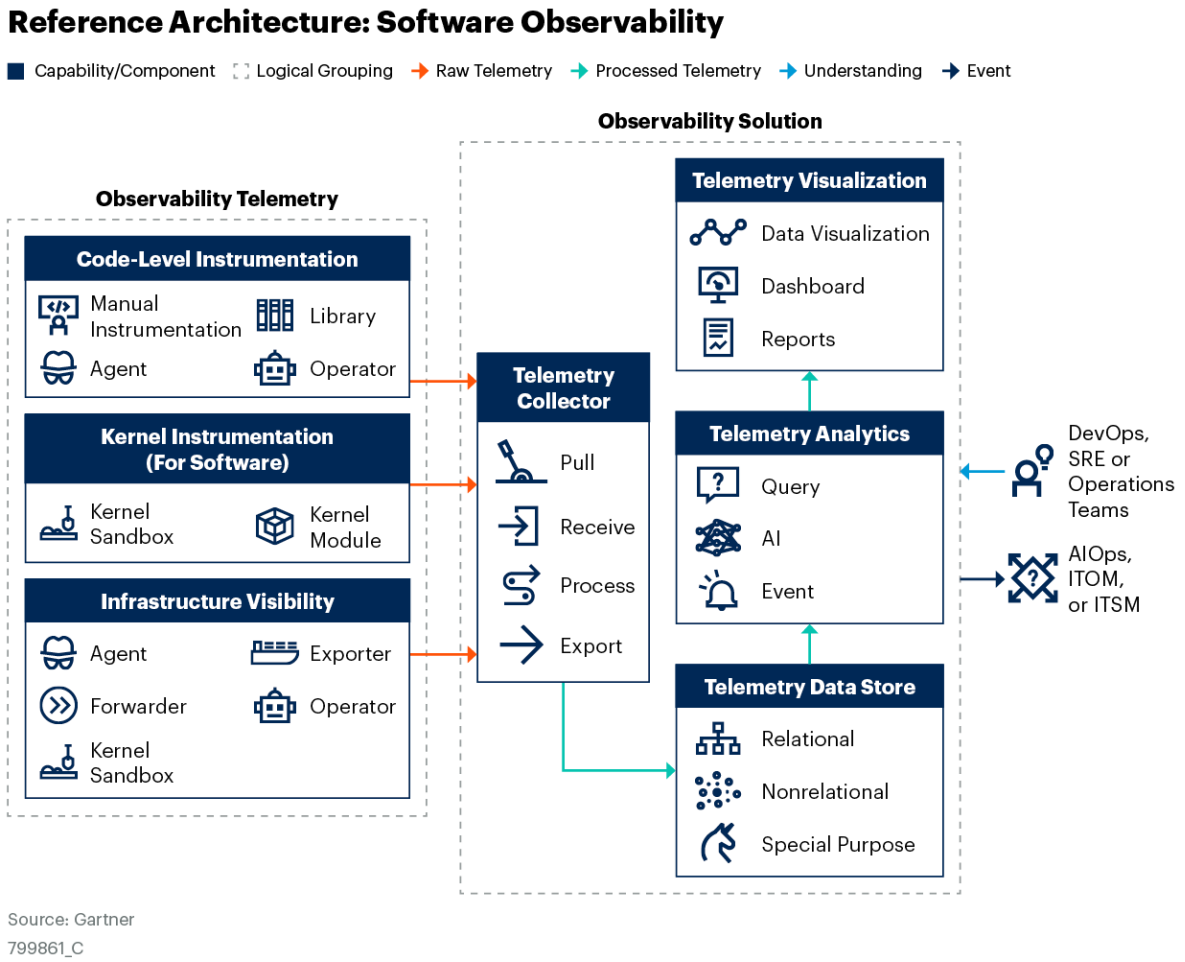
In the past, application performance monitoring (APM) was the standard mechanism for providing deep diagnostics of application health and performance. APM leveraged proprietary agents to instrument applications at runtime, collecting transaction traces and various application and related infrastructure metrics. As applications became distributed, asynchronous and more complex, developers and operations engineers recognized a need for a broad and targeted set of telemetry to allow for insight into the unknown unknowns — those business-impacting unanticipated issues unseen by traditional monitoring. The need for broad telemetry and related context led to the implementation of methods for collecting distributed traces, associated metrics and logs.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

The effort ultimately culminated in OpenTelemetry (OTel), a vendor-neutral collection of APIs, software development kits (SDKs) and other tools for code-level instrumentation. Although OTel can "generate, emit, collect, process and export" telemetry, it does not analyze or visualize telemetry data. [2] As of this publication date, most vendor-supported solutions and open-source projects that fall under APM and software observability categories have adopted OTel.

Traditionally, APM solutions provided the mechanism to collect telemetry, usually proprietary agents, and the solutions for analyzing the collected data. Today, open-source frameworks and mechanisms such as OTel have decoupled telemetry collection and analysis. Figure 9 illustrates the relationship between telemetry generation under the Observability Telemetry section and analysis under the Observability Solution section.

Figure 9: Reference Architecture: Software Observability



This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

In Figure 9, if you refer back to the telemetry life cycle (i.e., collect, ingest, analyze, act), the capabilities labeled code-level instrumentation, kernel instrumentation, infrastructure visibility and telemetry collector would fall under “collect.” The telemetry collector and telemetry data store would also fall under “ingest.” The telemetry analytics and visualization capabilities would fall under “analyze,” with the event data generated by analytics leveraged in “act.” Later on, the Step 3: Analyze Telemetry for Resolution section will dive more deeply into the areas of analyze and act.

Since OTel is now the industry standard for code-level instrumentation, consider its adoption when working toward optimizing for monitoring and observability. The following are the strengths and weaknesses of implementing OTel in your organization.

Strengths

- **Industry-standard:** The wide adoption of OTel is leading to its telemetry data models becoming the standard for new technologies while supporting the mapping and transformation of other telemetry sources.
- **Vendor-neutral:** You can now instrument your organization’s code without depending on proprietary agents. Telemetry data ownership remains with your organization as it is not locked into a vendor-specific format. You can leverage any observability platform that supports OTel for analysis and visualization and migrate to other telemetry analysis solutions without reinstrumenting your code.
- **Analysis-driven:** As OTel has shifted the development of telemetry collection from proprietary to open source, APM and observability vendors have shifted their focus and investment to analysis and visualization, providing organizations with more actionable data.

Weaknesses

- **Development support:** The successful implementation of OTel requires development support and resources. Although autoinstrumentation continues to grow for supported languages, you must consider some manual instrumentation to increase granularity and relevance for obtaining maximized value from observability.
- **Data volume:** True observability requires a great deal of data. OTel will generate a high volume of data that needs to be stored and processed. When leveraging OTel, you must strategically invest in data storage and management.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **Evolving standards:** Although OTel has dramatically matured since 2020, it is still relatively new. OTel has been widely adopted; however, vendors are still transitioning between their proprietary agents and supporting OTel. This disruption in the marketplace continues to lead to competing priorities.

For more information on OTel and APM, see Assessing OpenTelemetry's Importance to Application Performance Monitoring.

Kernel Instrumentation (for Software)

OTel is not the only open-source method for collecting observability telemetry on the market. Another promising technology, extended Berkeley Packet Filter (eBPF), enables low-level OS visibility and offers the promise of broad observability coverage by deploying sandboxed programs securely within the Linux kernel. eBPF is not a competitor to OTel but rather an automatic means of capturing observability telemetry without time-consuming implementation.

eBPF can supplement the telemetry types of distributed traces, metrics and logs with two other observability telemetry types: dynamic tracing and continuous profiling. Unlike distributed tracing, dynamic tracing does not provide the end-to-end path of an application request but rather detailed diagnostic information about a program's execution. Profiling is a method of collecting application performance data and resource consumption. Historically, profiling was resource-intensive and done manually as a diagnostic tool to solve an application problem. Continuous profiling differs because it is regularly collected at intervals and has a low impact on application performance.

For more information on kernel instrumentation, see Reference Architecture Brief: Software Observability.

Step 2.3 Monitor Infrastructure

In the past, it was common to think that monitoring infrastructure represented the entirety of monitoring. However, as noted earlier, infrastructure that appears performant does not necessarily deliver a good end-user experience for applications leveraging that infrastructure. In addition, software abstraction from the underlying infrastructure makes it more difficult to correlate infrastructure monitoring with application health.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Nonetheless, these factors do not make infrastructure monitoring irrelevant for two reasons. First, software observability requires traditional monitoring for infrastructure visibility to understand the impact of infrastructure on applications. Second, teams that support infrastructure (or domain) stacks require domain-specific monitoring for their roles. This research focuses on the former reason for resolving SLO-impacting incidents quickly or preventing them from occurring.

For this research, infrastructure monitoring includes the health and performance of these domains:

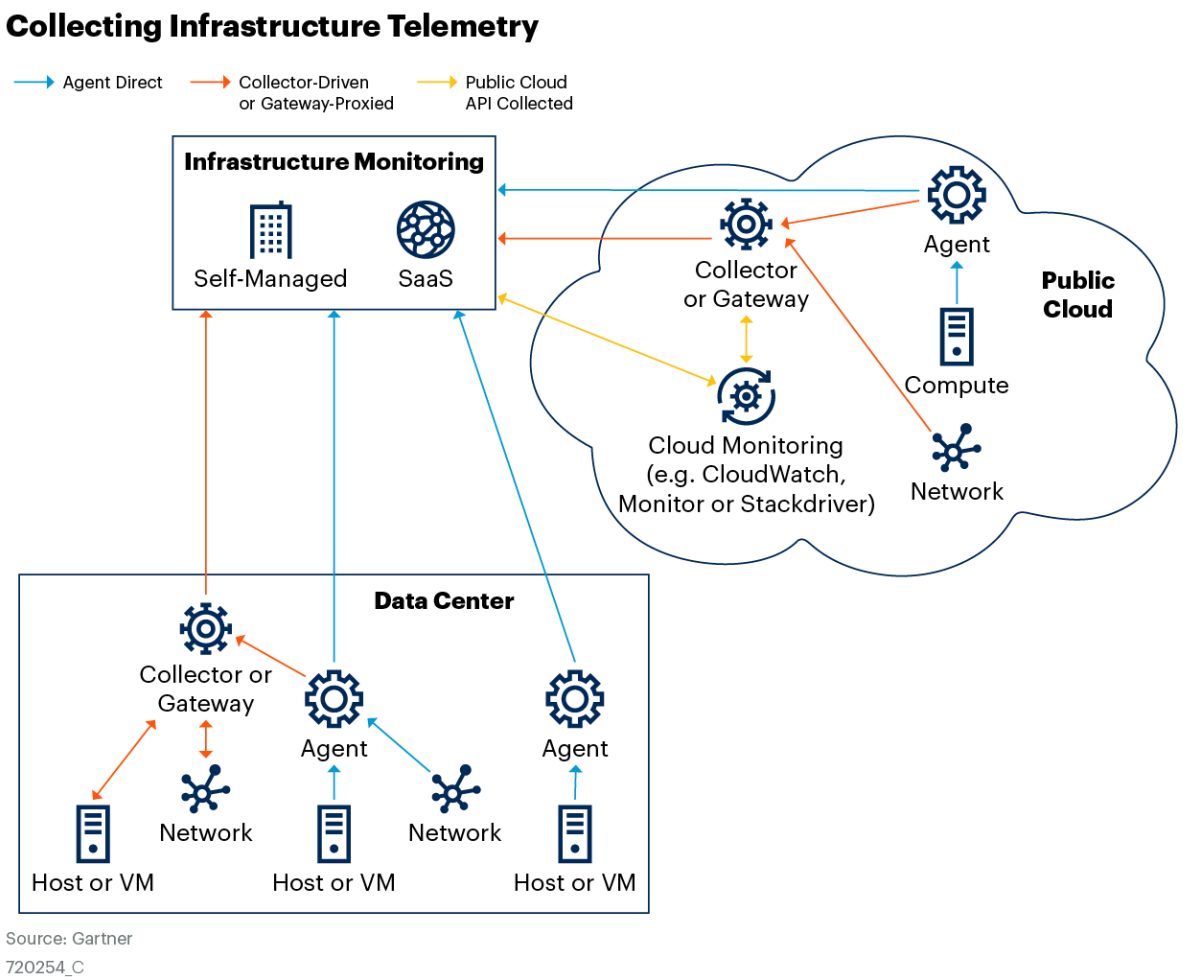
- Servers (virtual and physical)
- Storage
- Networks
- Hypervisors
- Public cloud
- Containers

There are various ways to collect telemetry from monitored infrastructure into a telemetry analysis solution. Figure 10 illustrates the most common ways:

- **Agent:** Agents typically reside on (e.g., VM) or near infrastructure and perform actions (e.g., commands) that generate and send telemetry upstream to a telemetry analysis solution, whether SaaS or self-hosted.
- **Collector-driven:** This agentless solution requires deploying collector software at an appropriate granularity to support the collection and aggregation of telemetry from monitored devices.
- **Gateway proxied:** This superset of the collector-driven method supports agentless collection for devices that cannot or do not run agents, but the collector can proxy telemetry upstream from locally installed agents.
- **API-polling:** There are many ways to instrument the public cloud. Some monitoring solutions treat IaaS as any virtualized compute resource and use agents or collectors as telemetry sources. More modern solutions include collecting telemetry from cloud provider monitoring services and APIs directly, which can be done in place of agents or in addition to them.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 10: Collecting Infrastructure Telemetry



Prioritize these criteria in the selection of your infrastructure monitoring technologies:

- **Current capabilities:** Ensure that your current and near-term hardware, software and service assets are supported and that sufficient extensibility exists to mitigate changes in direction (e.g., additional cloud providers).
- **Hosting model:** Switch to a SaaS-based telemetry analysis solution to reduce administrative overhead and ensure your mean time to detect (MTTD) remains unaffected. A self-hosted telemetry analysis solution may face performance issues or downtime during more significant incidents, potentially impacting your MTTD.
- **Collection preferences:** Implement a clear architectural strategy on the technologies deployed for the collection of infrastructure telemetry to avoid duplication and support reliability for telemetry collection.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **New solutions:** Telemetry analysis solutions continue to grow in capability and complexity. Although additional capabilities might be attractive, verify that any potential tools you investigate will adequately meet your needs.

Log Data

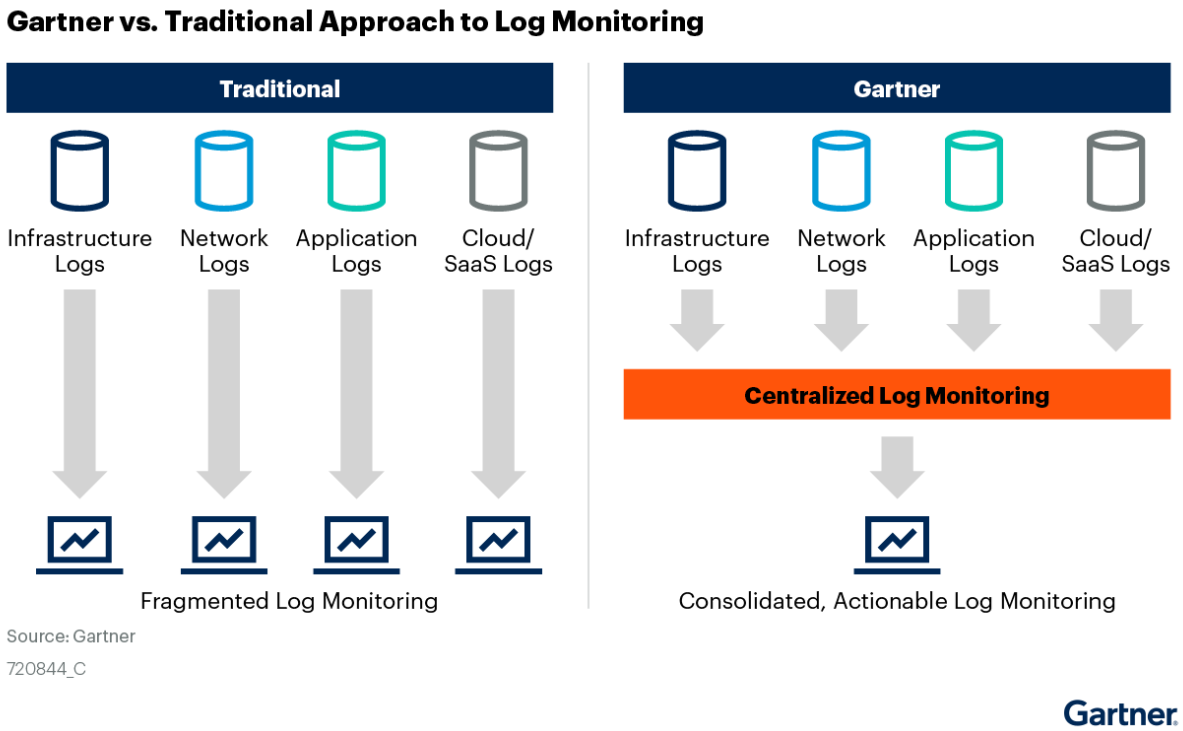
Logs are critical to monitoring and observability. Everything you wish to monitor or understand generates logs. Unfortunately, not all log data is valuable, or its value may diminish once the related business activity successfully ends. In other cases, logs provide the missing link that allows you to identify or resolve a longstanding defect. Logs may need to be retained for years for regulatory or compliance reasons. This diversity in source, value and purpose makes consistent log monitoring a formidable challenge.

Earlier in this research, a log was defined as a text string or structured data, a time stamp, and metadata. It's worth adding that a log is also a collection of messages, usually line-delimited. Those messages are not necessarily related to each other. Structured logs have all the characteristics just described, but the contents are represented as a set of key-value pairs, such as JSON. Events from a data management perspective are essentially the same as structured log data.

Logs serve multiple purposes. Log data serves as monitoring telemetry. It is also essential to observability having a vital role in troubleshooting and root cause analysis. In Guidance Framework for Deploying Centralized Log Monitoring, Gartner provides detailed recommendations for using and managing log data for monitoring. Figure 11, from that research, differentiates Gartner's recommended centralized monitoring from the traditional way many organizations may be handling log monitoring.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 11: Approaches to Log Monitoring



The variety of log formats complicates establishing a consolidated strategy. Text logs compress well, but compressed logs are difficult to index and search. As key-value pairs, structured logs provide near-infinite transport and size flexibility, but indexing high-cardinality fields is costly and makes substantial demands on analysis and reporting software. Different log sources often use inconsistent identifiers for the same fields, suggesting a transformation process is necessary for ingestion. Seemingly, the adversity keeps coming.

Fortunately, log monitoring solutions are available to minimize the impact of these challenges. The following are examples of vendors providing log monitoring solutions:

- ChaosSearch
- Datadog
- Dynatrace
- Elastic
- Honeycomb
- Grafana Labs

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- LogicMonitor
- Logz.io
- Mezmo
- New Relic
- ScienceLogic (formerly Zebrium)
- Splunk
- Sumo Logic

Telemetry Transformation and Transport

Logs have generally been collected by mechanisms different from those used to collect time series data, although the growth of multi-instrumentation repositories and growing telemetry volume are changing that approach. Log-specific forwarders like Fluent Bit are still prominent, but the OTel collector can receive logs in various formats or in its native OpenTelemetry Protocol (OTLP) along with traces and metrics from multiple forwarders. This single-collector approach allows for enriching and transforming telemetry data to improve the quality of data sent to telemetry analysis solutions for correlation and analysis.

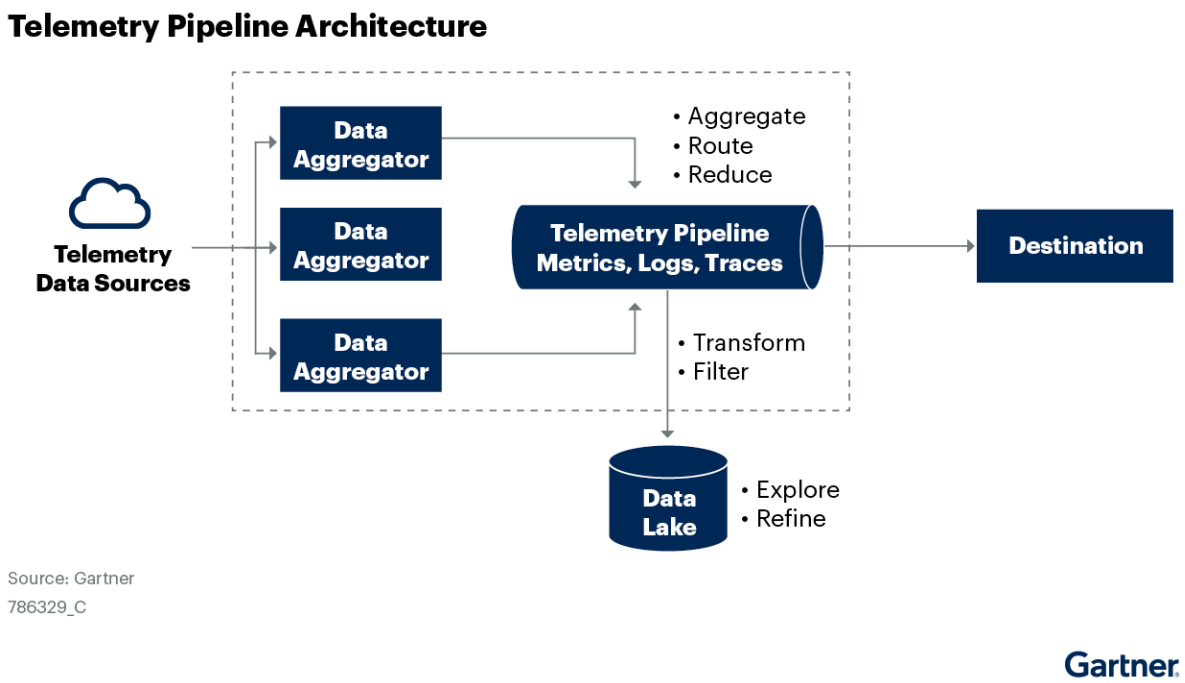
The need for telemetry data enrichment and transformation, along with the explosion in volume brought about by the growth in observability, has led to the emergence of telemetry pipelines. The purposes of telemetry pipelines are to:

- **Transform** telemetry data into a format consumable by the destination telemetry analysis solution.
- **Enrich** telemetry with context to improve the insight gained from analysis.
- **Reduce** redundant telemetry from being ingested into destination telemetry analysis solutions.

Figure 12 provides a high-level view of telemetry pipeline architecture.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 12: Telemetry Pipeline Architecture



When you have implemented the monitoring and observability reference model, investigate the potential of a telemetry pipeline to address your critical issues. If your top challenges include contextualizing telemetry for observability or streamlining the transmission and analysis of telemetry, these pipeline solutions can be a powerful tool. In environments with unmanaged and excessive telemetry transmission, telemetry pipelines might offer the rare combination of improved quality and lower costs.

For more information, see Enhance Monitoring and Observability via Telemetry Pipelines.

Read More

Enhance Monitoring and Observability via Telemetry Pipelines

Guidance Framework for Deploying Centralized Log Monitoring

How to Monitor Containers and Kubernetes Workloads

Reference Architecture Brief: Software Observability

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Step 3: Analyze Telemetry for Resolution

Step 3 activities focus on methods and technologies supporting the telemetry life cycle's analysis and act portions. As this research focuses on monitoring and observability, these activities ensure the context is in place to continuously analyze incoming telemetry, identify notable events and initiate response protocols when appropriate.

As identified in Step 1, the focus of telemetry analysis must be on supporting business objectives. Telemetry analysis aims to prevent issues before they impact the business or to resolve them quickly to mitigate their impact. The telemetry analysis function also includes impact-based prioritization of incidents.

Telemetry Analysis Solutions, Event Intelligence Solutions and AIOps

Until now, this research has mainly referenced telemetry analysis solutions (i.e., monitoring and observability solutions) as those that ingest monitoring and observability data for analysis. It has focused on metrics, logs, traces and referenced events. For monitoring and observability, events are typically generated by telemetry analysis solutions. Events can also be a telemetry data type leveraged for monitoring and observability. For example, SNMP traps from IoT devices can describe and identify an impacting state change (i.e., an event).

Telemetry analysis solutions most often consume metrics, traces, logs — along with other types of telemetry (e.g., packets, flows, profiles) — and even events. Step 1 covered the event intelligence solutions that primarily consume events. Telemetry analysis solutions tend to be stand-alone solutions. Event intelligence solutions may be stand-alone or components of another platform. In the past, those other platforms may have been ITSM systems that added event management as a valuable feed for incident management. Expanding those event management components to include analysis and AI was a natural evolution.

Event intelligence is event management augmented with multifaceted AI, correlation, advanced analytics and learning capabilities.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

It is important to note that stand-alone event intelligence solutions have long been an important part of the monitoring and observability marketplace. Products like BigPanda and Moogsoft help correlate, minimize and analyze the large volume of events from multiple monitoring solutions and other event sources. In the past, we described these tools as AIOps platforms.

As the importance of analysis and AI grew to support observability, many telemetry analysis solution vendors began adding AI features to their solutions and tried to leverage their broader monitoring platform capabilities to market their solutions as AIOps platforms. This created confusion between tools with embedded AIOps features and AIOps platforms. For this reason, at least, Gartner is retiring AIOps platforms as a market and creating greater differentiation by referring to these as event intelligence solutions.

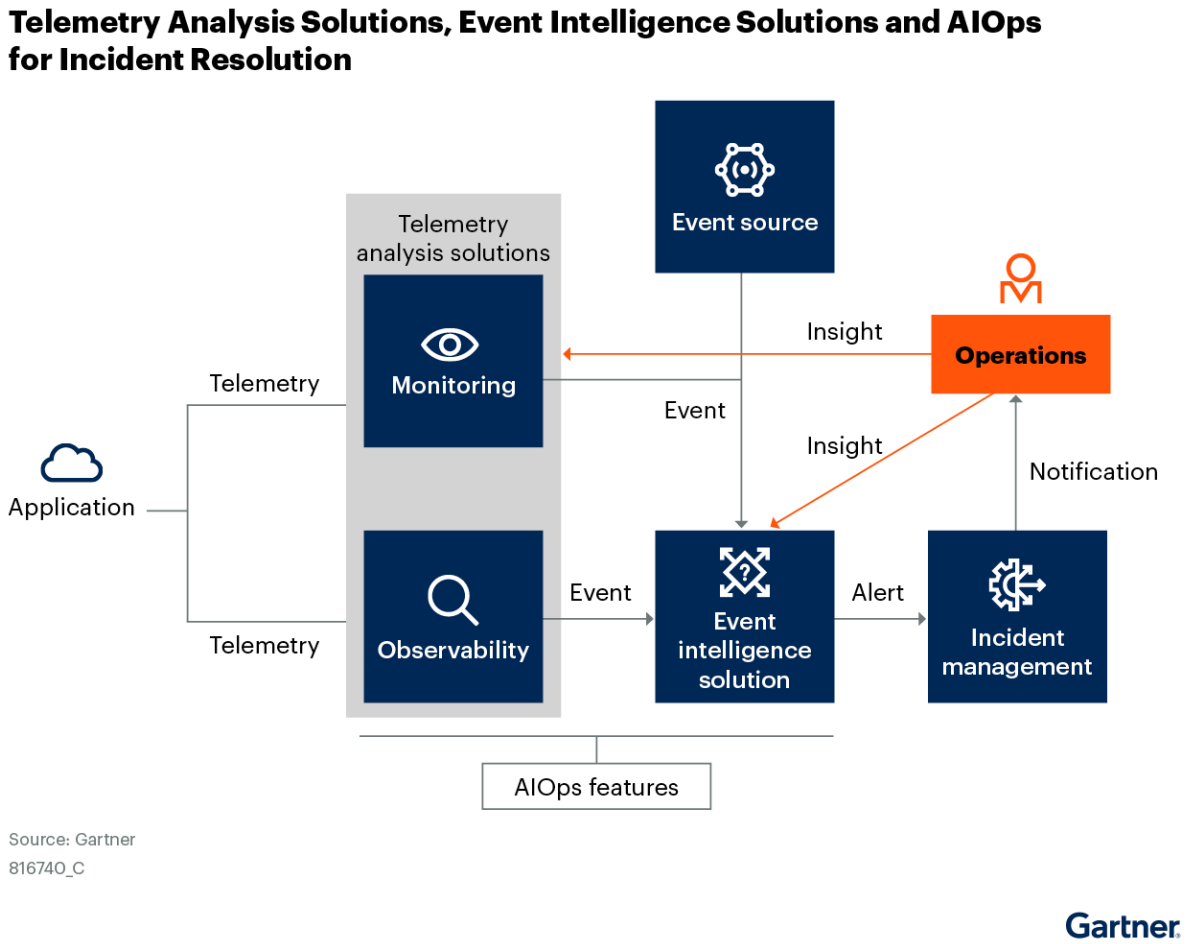
For the remainder of this research, AIOps will refer to the general capability of advanced analysis and AI applied to gaining insight from telemetry (e.g., metrics, traces, logs and events). Event intelligence will refer to those solutions, whether stand-alone or part of a more extensive offering, formerly described as an AIOps platform. AIOps features are enhancements to the existing functionality of monitoring and observability solutions that don't change the role or nature of the tool. They enhance the tool's ability to identify seasonal or subtle patterns that are more nuanced than static thresholds. Understanding that AIOps features exist in telemetry analysis solutions and event intelligence solutions is essential.

For more information, see Solution Criteria for AIOps Platforms.

Figure 13 clarifies the relationship between telemetry analysis solutions, event intelligence solutions and AIOps features in analyzing telemetry for resolution.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 13: Telemetry Analysis Solutions, Event Intelligence Solutions and AIOps for Incident Resolution



Step 3.1 Identify Impacts

Identifying business-critical impacts requires understanding included services and the level of impact on those services. This service-based insight must have enough metadata about the individual entities that comprise a service and the relationships between them to associate an event with services (i.e., topology). In some cases, the master representation of these relationships and dependencies is in a CMDB, and the service discovery process collects that data using the CMDB data as a starting point.

Service Topologies

The customer-facing services to which your SLOs have been attached are built upon a multitude of additional technologies (e.g., computer, storage, network, container and cloud) that must be monitored themselves. Within domain-specific telemetry analysis solutions, infrastructure monitoring and software observability rely on building and maintaining an understanding of the relationships between monitored entities.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Monitoring solutions can discover these relationships for infrastructure but may require manual adjustment or programmatic enrichment in highly dynamic or complex scenarios. Software observability telemetry (e.g., traces) such as that generated by OTel can be leveraged by telemetry analysis solutions to map the relationships between discovered software services. Some telemetry analysis solutions can show the relationship between software service configuration items (CIs) and related infrastructure. Others may only visualize the relationships between software services but provide infrastructure relationship information through drill-downs. Integrations between telemetry sources can feed discovered data such as CIs to your organization's CMDB to leverage service topologies with incident management.

Why Is Topology Necessary?

Telemetry analysis solutions and event intelligence solutions receive and process events from other domains. Therefore, context about the source of the events allows the algorithms to more accurately characterize the business impact. In addition, it is not uncommon for the failure of a single component to result in numerous, sometimes thousands of related events from other monitored entities. To associate these events with — and correctly identify — the source of the problem, the relationships between entities are required, which usually takes the form of a topology.

At one time, the CMDB promised to be a single source of truth for all the CIs and their relationships. In reality, there were challenges to this promise:

- Kubernetes, containers, serverless and other cloud technologies create dynamism in infrastructure that can impact the accuracy and quality of CMDB data.
- A central CMDB (e.g., content management system [CMS]) that receives downstream data may not contain the granularity of data needed for telemetry analysis.
- Regular automated CI pruning leveraging priority rules is required to avoid duplication in a central CMDB. This pruning may unintentionally remove critical CI data needed by downstream telemetry analysis solutions or event intelligence solutions for analysis.

Monitoring and observability telemetry analysis solutions help to address these challenges:

- Integrations with telemetry analysis solutions provide access to discovered CIs, such as containers and other ephemeral instances, for populating a central CMDB.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- Many telemetry analysis solutions will assemble service topologies from the telemetry they ingest for analysis, preventing them from having a critical dependency on a central CMDB.

Telemetry analysis solutions actively manage the critical data needed for root cause analysis and serve as a CI discovery source for a central CMDB. Telemetry analysis solutions can improve CMDB data quality and set expectations for an upstream CMDB typically deployed to support ITSM activities. It is a standard best practice to send only valuable and needed aggregate data upstream. The same practice is proper for CIs and associated metadata.

Telemetry analysis and event intelligence solutions should include service topology assembly and mapping or some approach to describing the relationships between systems and system events to enhance analysis. Topologies allow insight into the way requests flow between services, the dependencies between services and the dependencies between services and infrastructure. In addition, topologies serve as a way to identify where anomalous behavior may occur or to help identify whether that behavior is the result or cause of an impacting event.

Table 6 provides examples of telemetry analysis solutions and event intelligence solutions that include service topology technologies.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 6: Service Topology Technology Examples

(Enlarged table in Appendix)

Vendors	Solution type	Technology
BMC	Event intelligence	Service Mapping
BigPanda	Event intelligence	Real-time Topology Mesh
Cisco (AppDynamics)	Telemetry analysis	Flow maps
Datadog	Telemetry analysis	Service Map
Dynatrace	Telemetry analysis	Smartscape
Elastic	Telemetry analysis	Service Map
Grafana Labs	Telemetry analysis	Application Observability service map
Honeycomb	Telemetry analysis	Service Map
LogicMonitor	Telemetry analysis	Application Topology Map
Logz.io	Telemetry analysis	Service Map
New Relic	Telemetry analysis	Observability Maps

Source: Gartner

Anomaly Detection for Performance Metrics

The best way to resolve a business impact is to prevent one. Often, organizations require their operations architects to implement predictive monitoring without understanding what it entails. Predictive monitoring leverages the analysis of historical data to look for significant deviations or anomalies and anticipate potential impacts before they occur.

Tracking changes in performance metrics is an example of anomaly detection. Both monitoring and observability capture performance metrics. Examples of performance metrics for host monitoring could include CPU, disk and memory utilization. For software observability, it could be the typical response time for completing a service request. For DEM, it could be the completion time for page load time.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

By tracking the normal performance of these metrics over time, a telemetry analysis solution can identify meaningful deviations and generate an event. You may think deviations in the examples cited above would be larger CPU, disk or memory utilization; slower service requests; or longer page loads. However, this may not be the case. More disk space could mean that an application is unable to write files. Quicker service requests could mean that other back-end services are not engaged. Faster page loads may mean that essential images are unable to render.

Tracking and identifying anomalies is a critical AIOps feature that can detect problems before they impact the business or an end user has to bring them to the attention of I&O. Anomaly detection also brings additional value to the types of metrics that you have been capturing for years by looking at the data differently. This type of analysis is important as you consider user journeys and SLOs.

User Journeys

Step 2 introduced SLIs, SLOs and the apparatus needed to collect the experienced-based telemetry to inform them. Step 3.1 covers understanding how to use them to monitor critical business activity in the form of user journeys.

You may already have a cataloged collection of user journeys if your telemetry analysis solution includes business activity monitoring or an integrated method for defining SLIs for SLOs. You may be able to perform much of your SLO tracking within those solutions directly. Or, if you have a DEM solution that does STM, you may have already gone through such a user journey collection and prioritization process. In that case, you have at least the starting point — and perhaps much more.

Development's Role in Instrumentation

To leverage software observability telemetry for SLOs, use OTel auto-instrumentation (e.g., libraries, agents, operators) in applications developed by your organization. Auto-instrumentation will provide most, if not all, of the data you need. Add manual instrumentation only if additional granularity is required. Regardless of technique — structured logs, metrics or distributed tracing — rich user experience instrumentation is the responsibility of the product development team. Historically, the deployment of instrumentation by operations teams has been suboptimal as it is challenging to "bolt" on instrumentation in production.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Operations plays a critical role in supporting the telemetry collection and ingestion mechanisms and providing the telemetry analysis solution. However, development must implement instrumentation and give the business and empirical understanding needed to define SLIs for SLOs. Historically, operations drove APM; however, software observability must be a cooperative effort between development and operations. Shifting the instrumentation of software left ensures that SLIs and deep, unit-of-work-based telemetry is available for SLO tracking.

For more information, see Assessing OpenTelemetry's Importance to Application Performance Monitoring and Reference Architecture Brief: Software Observability.

Performance Budgets

Balancing new features with their impact on performance is challenging. When tracking SLOs, there is a risk of incomplete or unrealistic performance testing on a preproduction basis. This failure to test can lead to the new release being unable to meet an established SLO. Incorporate a performance budget to guide the compromise and consensus necessary to ensure services remain compliant. Performance budgets represent a framework that product owners, developers, SREs and operations agree to use to manage the impact of evolving capabilities on user experience over time.

Do not limit performance budgets to production systems. Instead, extend budgets to development environments to detect budget violations sooner. Determining a budget strategy requires understanding the system's performance characteristics throughout its life cycle. As with other defects, the further along the release cycle defects are identified, the more costly they are to repair. Validating performance budgets should be a gate in your CI/CD pipelines. Validating every build enables faster feedback to developers and helps narrow down the exact commit point that violated the budget.

For an example of the benefits of a performance budget, imagine a product owner who adds more images to the homepage to increase revenue on an e-commerce application and then balances the performance budget by removing additional JavaScript functionality.

Identifying and Monitoring User Journeys

Here are the steps in the process:

1. Enumerate the user journeys through your applications and services.

2. Order the user journeys by business impact.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

3. Use the guidance in Step 1 to define the SLIs that track the user experience.

4. Establish SLOs, performance budgets and measurement periods.

5. Build reporting and trending dashboards for SLIs, SLOs, performance budgets and error budgets.

6. Create rules to generate events for SLO violations and errors or performance budget consumption.

Getting started is the most difficult step. There is a natural inclination to get bogged down in “most critical” or find the application you intended to begin with difficult to monitor because of hosting or platform limitations. Instead, start with the user journeys most easily understood and convenient to monitor.

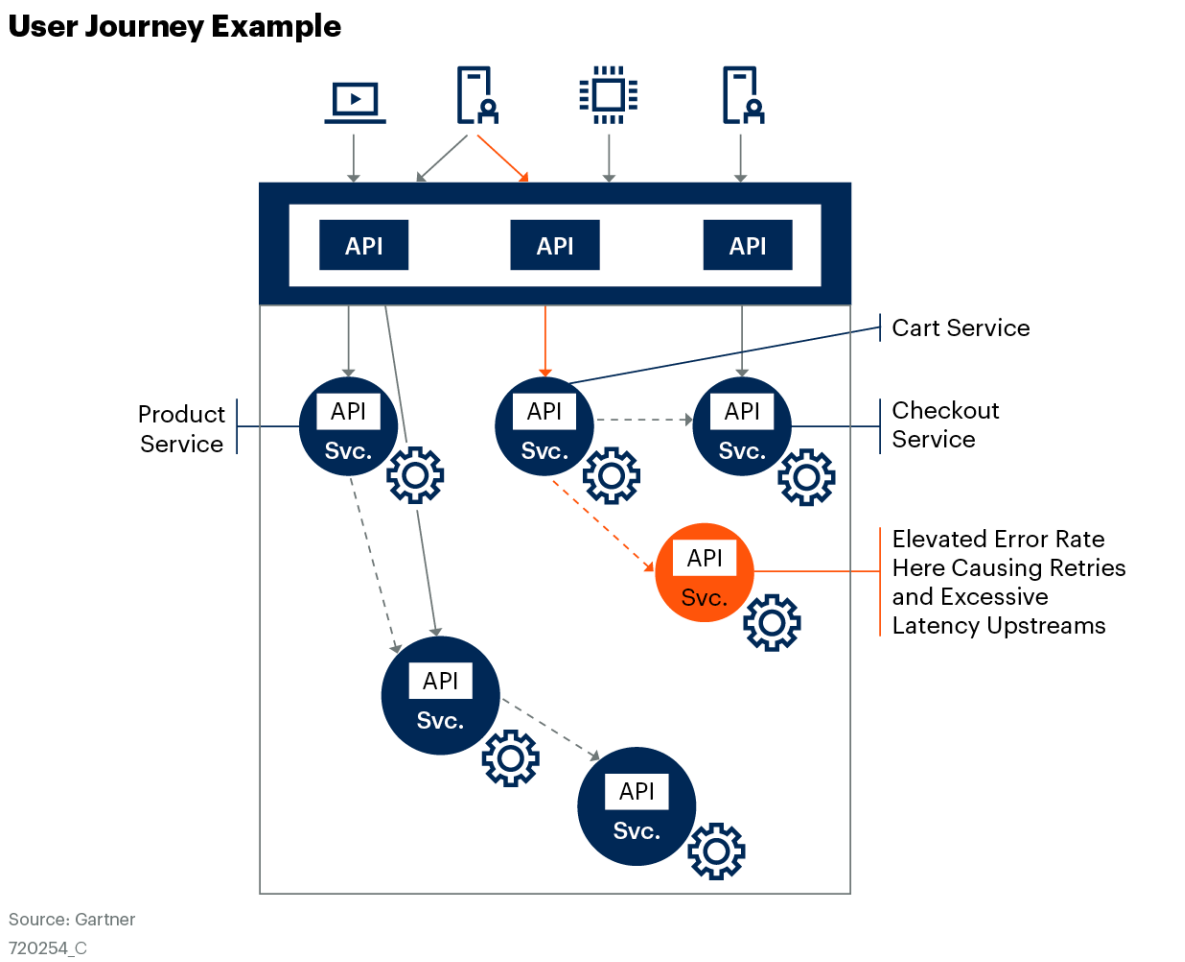
An example might help. In a retail or online commerce environment, the common user journeys could be:

- User authentication and loyalty profile retrieval
- Catalog traversal and product details retrieval
- Check out and complete purchase

These are “critical” journeys from the user’s perspective and arguably interdependent. The last one, however, is revenue generating and, therefore, the most business-critical user journey of the three. Figure 14 illustrates how a problem in an inner service can spread to threaten the SLO on a critical user journey.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Figure 14: User Journey Example



Latency and availability SLIs are the right way to measure and report on this critical user journey. There are a few ways to set initial targets. Recall that one of the most critical attributes of SLOs is that they are achievable. If enough historical data is available to set a starting point, start with that.

Reporting and alerting on SLOs and error budgets may be straightforward if your organization uses a telemetry analysis solution with built-in support for SLOs.

Finally, there are events. In addition to the thresholds, consider generating events for the error budget burn rate to ensure you have time to recover before exhausting the budget.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

> **Establishing SLOs for critical user journeys may appear foreign, but it will transform your product development and release processes. Value-driven transformation is a principal objective of this Solution Path.**

The following are telemetry analysis solutions that support SLO tracking:

- Amazon CloudWatch
- Elastic Observability
- Grafana Cloud
- Honeycomb
- IBM Instana Observability
- Splunk Observability Cloud

Step 3.2 Determine Root Causes

Although it is crucial to identify existing or potential business impacts, identifying the underlying cause of these problems is equally critical. This practice is called root cause analysis and has historically been difficult. Even though an issue might be detected, the actual cause may not be apparent. Finding the root cause has traditionally been time-consuming, involving manually scrutinizing events and logs from applications and infrastructure.

Today, AIOps features in telemetry analysis solutions and event intelligence solutions process large amounts of telemetry to infer root causes and augment operations engineers' understanding of them. This section covers examples (not an exhaustive list) showing how some technologies help determine potential root causes.

Event Management

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

As mentioned earlier in this research, an event is a state change. In monitoring and observability, an event is potentially a business-impacting state change. It was noted that events can be a telemetry data source. During this telemetry analysis for resolution phase, events transition from being only a type of telemetry to also identifying incidents that require resolution.

This discussion involves the definition of a few essential terms:

- **Event:** A state change for a service (or other CI).
- **Alert:** A state change that creates a negative impact. This term can also be used synonymously with the alert notification process.
- **Incident:** An unplanned interruption to a service or a reduction in the quality of service.
- **Notification:** The way to make a user aware of an alert or an incident.

Although not "official," these definitions help to provide context as this discussion proceeds. These are common words in I&O and ITSM that have a variety of meanings and overlapping semantic domains.

Most organizations experience a high volume of events from telemetry analysis solutions and other nonmonitoring sources. If there is a one-to-one relationship between events and alerts, the resulting high volume of notifications for operations engineers can create alert fatigue. Alert fatigue, especially over time, can result in missed impacts, obfuscated causes and distrust of the tools intended to help solve problems.

Some of the technical solutions involved in event management are:

- **Event intelligence:** Previously known as AIOps platforms, these solutions use analytics and AI to understand and correlate events across domains, identify meaningful patterns, and augment responses to those patterns.
- **IT operations management (ITOM):** These solutions manage the provisioning, capacity, performance and availability of computing, networking, and application resources and delivery. Often, ITOM tools include event management capabilities to support the performance aspect of their work.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **IT service management (ITSM):** These solutions may act as a source of change and configuration data and escalate events into incidents to support incident management.

All these may be dedicated solutions or the capabilities of a more extensive solution.

Event Management Requirements

The analysis of events is central to event management. Solutions that support event management must "understand" an event's underlying cause and determine whether it requires further action.

To make such a determination, solutions require important event context such as:

- Affected services
- Dependencies
- Business impacts
- Severity
- Responsible parties

As referenced earlier in this research, understanding how events connect to a particular business service will require well-defined service relationships. These relationships can provide insight into how an event might result from a dependency that is in an error state. Although raw events will not contain all the information above, telemetry analysis and event intelligence solutions can use available data sources to enrich events with additional context for analysis.

AI and Analysis

Telemetry analysis solutions and event intelligence solutions employ various AI and analytics capabilities to identify the root cause of an event. This determination extends beyond analyzing events alone and encompasses other forms of telemetry, including metrics and logs. Figure 13, in the Telemetry Analysis Solutions, Event Intelligence Solutions and AIOps section, illustrates this concept, depicting the operations team member acquiring valuable insights from a telemetry analysis solution or event intelligence solution.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

The following are examples of AIOps features leveraged for understanding the root cause of an event:

- **Anomaly detection** identifies and highlights uncommon or unexpected changes or deviations in telemetry data, revealing a potential root cause for a subsequent event.
- **Event correlation** clarifies the relationship or connections between events from multiple sources via context, topology or other data, helping to identify the potential source event and suggest a root cause.
- **Pattern recognition** compares the known root causes of previous events with the analysis of current telemetry to determine the potential underlying cause of a current event.

For more information, see Infographic: Using AIOps to Manage Operational Telemetry and Solution Criteria for AIOps Platforms.

Change Management

Ironically, changes to improve performance and end-user experience may negatively impact what they seek to improve. Given the current operational paradigms, preventing or slowing down change is unlikely, but use changes and change failure rates to understand your environment better. Leverage event intelligence solutions with change management data to predict and prevent potential impacts before implementing changes. Event intelligence solutions can also help identify already implemented changes as the root cause of a current problem.

Step 3.3 Resolve Impacts

The goal of identifying impacts and determining their root causes is to resolve them. In an ideal scenario, the hope is to predict an impact before it occurs and prevent it. Ideally, if you detect a potential impact, you can avoid it. But, this still requires the resolution of the underlying cause. So, analyzing telemetry for resolution is always the goal of the activities and technologies discussed in this research.

The optimal resolution of impacts requires a combination of human skills augmented with AI and automation, which are available in AIOps features.

The Human Factor

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Many organizations use a tiered support structure. Level 1 (L1) support resources serve as first responders that follow manual incident response processes and escalate to Level 2 (L2) and Level 3 (L3) as required. The incident may originate from telemetry analysis solutions, event intelligence solutions or the service desk. L1 responsibilities include:

- Acknowledging alerts and incidents
- Conducting a preliminary incident assessment
- Obtaining suggestions from a knowledge base or AI assistant
- Identifying domain experts or responsible parties
- Following escalation policies

The main challenges organizations encounter with a manual incident approach are:

- High incident response times
- Poor incident communication and visibility
- Inability to meet DevOps agility or velocity requirements
- Lack of uniformity hindering process improvement

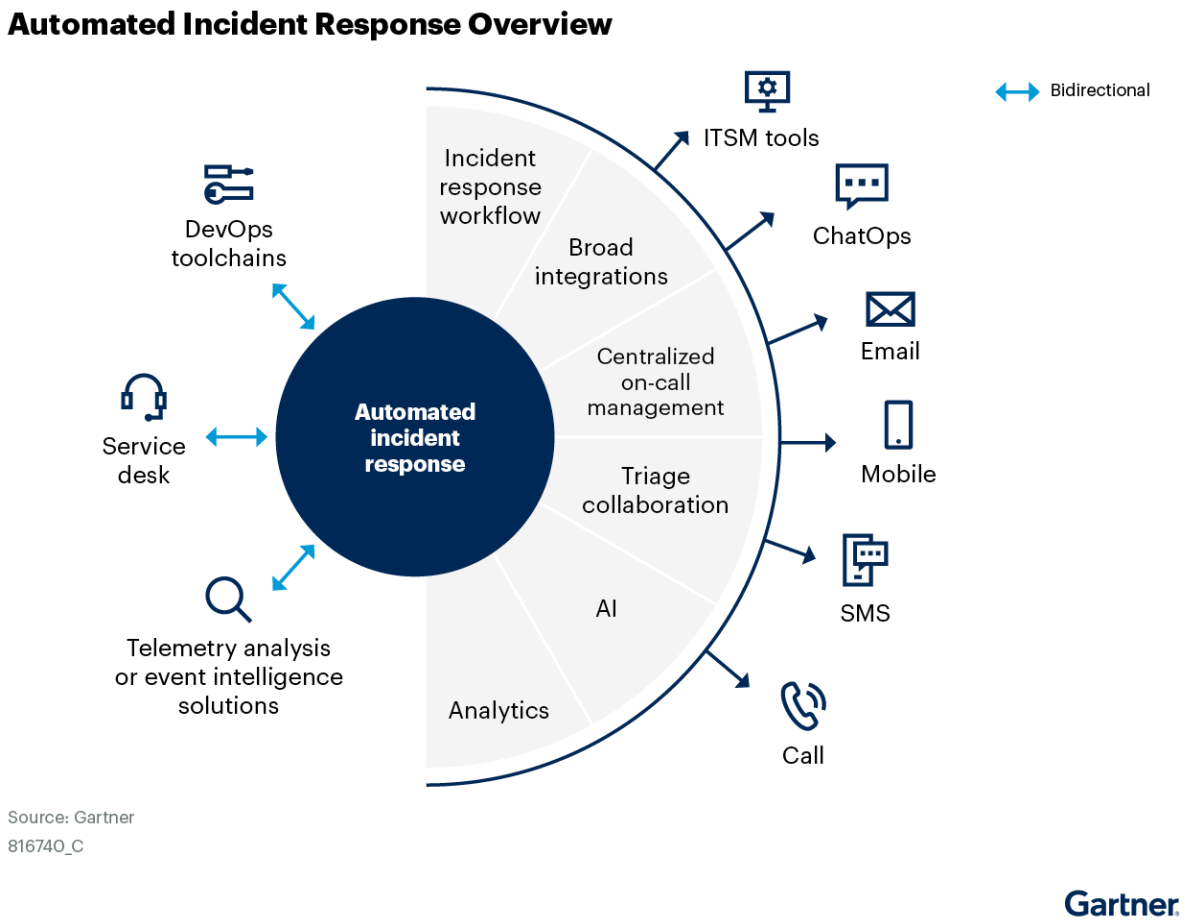
Automated incident response (AIR) streamlines the incident response process by automating some manual activities usually handled by L1 support. Focused initially on alert notification, modern AIR provides a focal point for incident responders with relevant context and tools necessary for triage. As shown in Figure 15, AIR solutions encapsulate these aspects of incident response:

- Incident response workflow
- Broad integrations
- Centralized on-call management
- Triage collaboration
- AI
- Analytics

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

For more details, see ITSM Best Practices: Automating Incident Management.

Figure 15: Automated Incident Response Overview



The following are general recommendations for organizations that decide to embark on automating incident response with AIR:

- **Establish a clear rating scale to prioritize incidents:** Develop consistent criteria for incident classification based on impact and urgency as they determine priority. Priorities are often ranked on a scale of 1 to 5, with priority 1 incidents garnering the most attention as they have the most significant business impact and require immediate attention. Work with the various business units to establish these criteria. This consensus-based set of criteria will help ensure incidents receive the correct level of attention. Once defined, many event management systems can automatically assign an initial priority level to the incident to trigger the appropriate incident response.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **Identify the actors and roles in incident response:** Incident response teams can be organized in a tiered or swarming model. A tiered structure allows incidents to be routed linearly through each tier until the incident is resolved. This structure is best suited for organizations with a high volume of low-complexity incidents. The tiers might be arranged as follows:
  - **Level 1:** Basic customer support
  - **Level 2:** Technical troubleshooting and support
  - **Level 3:** Expert support
  - **Level 4:** External support

Swarming support models, on the other hand, are interdisciplinary teams dynamically formed based on the characteristics of the major incident. These temporary teams collaborate throughout the resolution process and disband once the incident has been closed. This structure is best suited for a lower volume of highly complex incidents.

- **Leverage AIR ML, but optimize the event and incident detection points first:** AIR solutions provide AIOps features for noise reduction, but Gartner recommends addressing alert fatigue by combining noise reduction capabilities from telemetry analysis solutions, event intelligence solutions and AIR. Alert noise indicates a problem at the source, not the incident response layer. Leverage AIR for noise reduction as you optimize event analysis and incident detection with more accurate thresholds and notifications. AIR machine learning use cases include responder selection knowledge management.
- **Standardize alert actions across the organization:** When responders receive a notification, they can invoke specific actions based on incident workflow design configuration. When different teams design workflows, the alert action terminology might not necessarily follow the action definitions the responder intends. The responder should spend time resolving the incident rather than interpreting the workflow steps as they select an option on the notification. Address this challenge by standardizing the alert notification actions for responders across all incident workflows at your organization level. The development, DevOps, SRE and I&O teams should agree upon and then publish the definitions of responder actions.

Additional Guidance

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Select telemetry analysis solutions (e.g., monitoring and observability) and event intelligence solutions that integrate bi-directionally with your CMDB, striving for automated reconciliation, as static service topologies can quickly become obsolete. Document your event and incident response processes before automating them. The workflow in the systems you use effectively becomes your event and incident response process and should be codified externally for validation. It is even better if infrastructure and configuration as code capabilities are available to configure these systems.

Read More

Assessing OpenTelemetry's Importance to Application Performance Monitoring

Infographic: Using AIOps to Manage Operational Telemetry

ITSM Best Practices: Automating Incident Management

Reference Architecture Brief: Software Observability

Solution Criteria for AIOps Platforms

Step 4: Optimize and Refine

Until now, the steps in this Solution Path have focused on what to do now to bring monitoring, observability and event management into a position to resolve today's impacts. This final step considers how to build the foundation to meet future demands and maximize the frameworks and feedback loops.

Each event, alert and incident you experience is an opportunity to improve. The improvements could include reduced time to detect and reduced time to restore. Improvement might mean that an event is captured and remediated before its severity escalates to an incident. Or, on hopefully rare occasions, it might mean that the unexplainable slowdown affecting a small but vocal set of customers on alternate Tuesdays between 14:00 and 16:00 has finally been explained and resolved, and the fix pushed to production.

Although self-driving or autonomous digital operations may still be a goal for many of you, it is important to acknowledge the interim and attainable victories outlined in the previous paragraph. In your ongoing efforts to enhance your solutions and strategies, you should continuously work toward automating incident resolutions, determining visibility gaps and fostering a collaborative community.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Step 4.1: Automate Incident Resolution

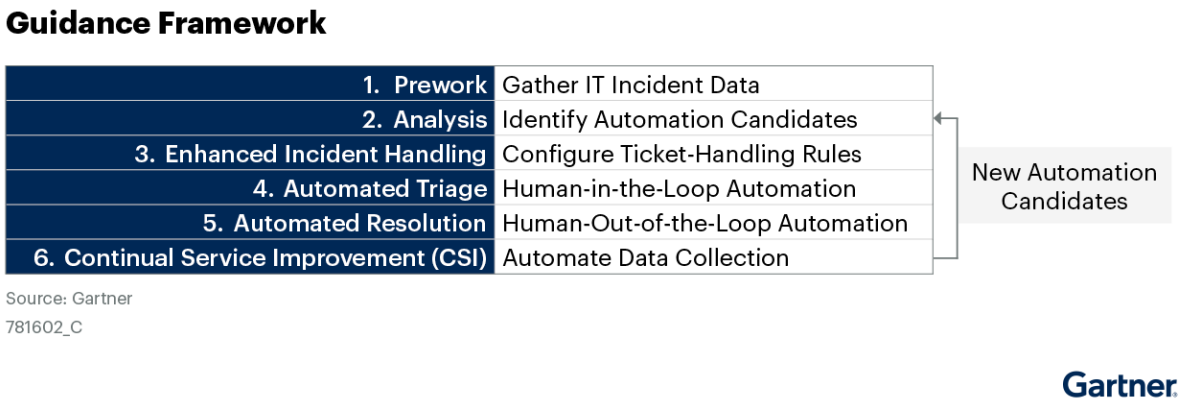
Step 3 addressed the capabilities of AIR. However, AIR is only one rung in the automation ladder for incident management. Automating incident resolution is an intentional and continuous process that leverages technology with human skills and understanding.

ITSM Best Practices: Automating Incident Management provides a multistep framework for the automation of incident resolution (i.e., management) that includes:

- **Prework** to gather incident management data to support the analysis of incident management.
- **Analysis** to establish a baseline for the current support services and identify opportunities for automation.
- **Enhanced incident handling** to optimize ticket handling and routing to reduce the time it takes to respond to and repair incidents.
- **Automated triage** for the development of partial automation to aid humans in expediting incident resolution.
- **Automated resolution** for developing full automation to increase the capacity of IT support teams while driving operational efficiency, speed and quality.
- **Continual service improvement** for managing automation and the ongoing optimization of incident management through analytics.

Figure 16 illustrates this multistep guidance framework.

Figure 16: Guidance Framework for Automating Incident Management



This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

> **Automating incident resolution isn't about saving money by cutting skilled resources. It is about delivering genuine value through quickly and successfully closing incidents. This kind of automation demands increased investment in skilled and knowledgeable people, not a reduction.**

For a deeper dive into automating incident management, see ITSM Best Practices: Automating Incident Management. For further study on incident management, see ITSM Best Practices: Implement a Major Incident Management Response Plan.

Step 4.2: Determine Visibility Gaps

Even though observability is at the heart of this Solution Path, you will always have visibility gaps. Issues will continue to arise that take you by surprise. However, rather than see these as failures, consider them as learning opportunities to "understand your lack of understanding."

Observability helps you understand the "unknown" unknowns, but you have to gather the telemetry required to get the insight you need. Your analytics and AI solutions can't provide the necessary insight if you don't have the correct data. So, some practical considerations exist for discovering and fixing your visibility gaps.

Postmortem Analyses

A postmortem analysis or retrospective is an opportunity to review and learn from a resolved incident or event. Here are some critical considerations for postmortems:

- **Focus on predicting similar events:** Postmortems should examine the incident and aim to improve your ability to predict and prevent similar problems in the future.
- **Conduct postmortems on more than major incidents:** Don't limit postmortems to major incidents alone. Use random sampling to review less critical incidents to determine whether the right data was collected to resolve the problem efficiently. This information can help to find visibility gaps that might otherwise result in a major incident in the future.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- **Foster a blameless culture:** It is essential to create an environment where blame is not assigned. Instead, encourage open and honest communication to address gaps and improve future incident response.
- **Be realistic about limitations:** Recognize that monitoring and observability have limitations due to cost, security, technology limitations or other external factors. Document these limitations and the resulting gaps. Review your limitations regularly to address them when possible.

Observability Feedback Loop

This Solution Path primarily focuses on preventing or resolving incidents; however, although the telemetry data and analysis collected are essential for avoiding or resolving incidents, they also offer valuable insights into performance improvements to enhance the development life cycle. The following actions are important for utilizing monitoring and observability data in development:

- **Incorporate observability data into your development pipeline** to make it an integral part of the process. This integration will allow developers to leverage telemetry data to predict and resolve problems while continuously improving application development and design.
- **Plan for software instrumentation in development** by making telemetry analysis solutions indispensable to the development toolbox. Telemetry analysis gives developers valuable insights into application performance and behavior, enabling them to proactively address issues, optimize code and improve application quality.
- **Share observability data and insights** to foster collaboration between operations and development teams. This partnership will reduce MTTR by ensuring both teams have insight into application behavior and performance. Collaboration facilitates faster problem resolution and continuous improvement across the development and operations life cycles.

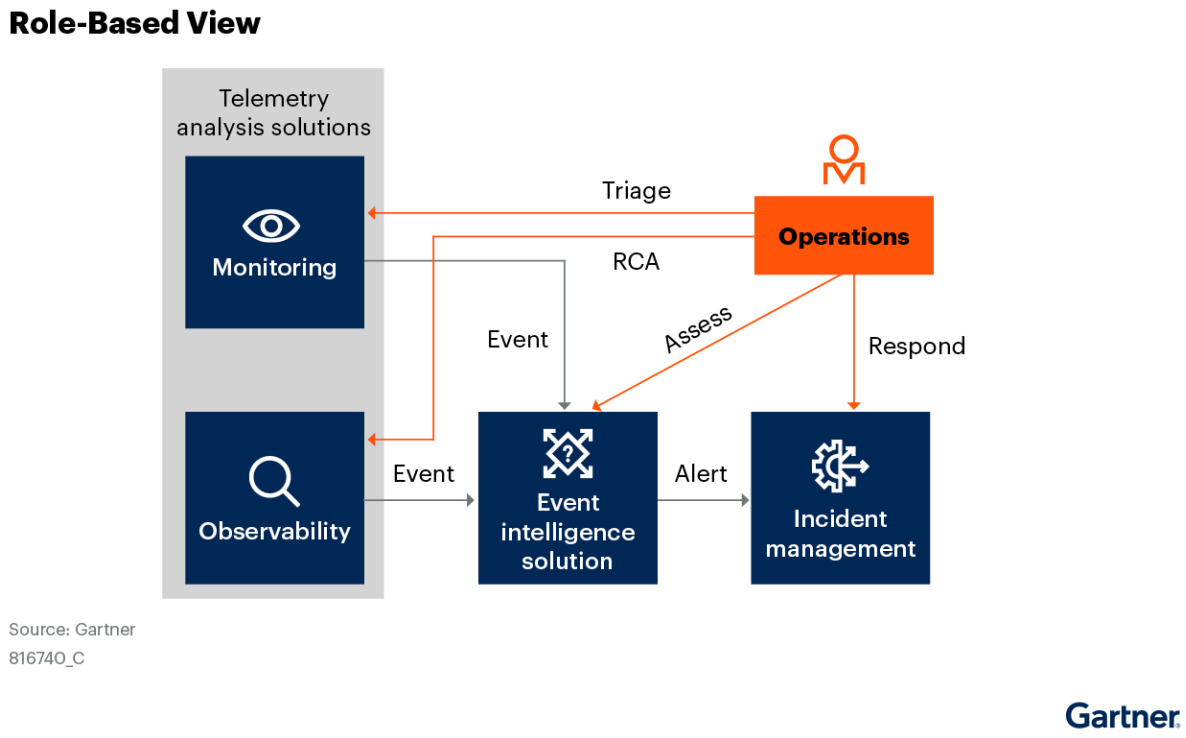
Role-Based Views

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

For this research, the term "view" includes dashboards, reports or any visualization method within a telemetry analysis solution or event intelligence solution. The usability of telemetry analysis depends on how data is presented visually. However, it's important to note that not all roles can use the same data in the same way. This Solution Path has outlined different integrated solutions for the telemetry life cycle — collect, ingest, analyze and act. Roles may use different solution entry points based on specific responsibilities and tasks.

For example, Figure 17 shows how an operations team member might use different solution entry points for various tasks based on their assigned work. In reality, roles typically use the same regular interface(s) for their duties. The specific entry points they use will depend on the type and level of data needed and the analytics or automation capabilities required to leverage that data.

Figure 17: Role-Based View



Successfully adopting the technologies described in this Solution Path requires evident utility. Utility means that the interfaces (e.g., dashboards and reports) presented to users provide valuable data to aid them in completing their tasks. Monitoring, observability and event intelligence solutions must help them solve problems efficiently. It is important to note that any perceived shortcomings in your solutions may not result from insufficient data but rather a lack of contextualized access to the data.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Here are some practical guidelines for role-based views:

- **Stick with out-of-the-box (OOTB) views whenever possible:** This may seem to contradict the idea of role-based views, but often, OOTB views deliver needed data. The vendors — or communities for open-source tools — that develop these solutions use the concept of a persona to define experiences and tailor the experience to a common persona across their customers. However, users may require training on how to navigate to the correct views or use them effectively. Before building a custom view, ask questions to understand why a current OOTB view can't meet the need for a role.
- **Design custom views for a role and not a user:** Custom views will be needed, but they should focus on supporting the scope of work for a role and not an individual user. We all have opinions, but standardized views for roles help codify best practices.
- **Build custom views collaboratively:** A solution owner is not necessarily the best person to design a view in isolation. Design and build views by teaming with the users who use them daily. Regularly check in with those users to see how their views support their work.
- **Be careful when allowing users to build their own views:** This will avoid the proliferation of unused views. Consider a process where certain role "owners" can build or update views for and with their respective teams. Regularly review the usage of views to determine whether some unused ones may be deleted.
- **Limit the number of visualizations:** Use the minimum number of visualizations required to understand the system's state. Technical teams should agree upon these visualizations and, more importantly, understand them thoroughly.

In the process of considering the need to determine visibility gaps, it has become apparent how essential teamwork and collaboration are to those activities. You need to focus on the importance of collaboration.

For more information on dashboards, reports and visualizations, see Reference Architecture Brief: Software Observability.

Step 4.3: Build a Collaborative Community

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

> **Optimizing monitoring and observability is about eliminating that 3:00 a.m. call for a problem you don't understand, let alone know how to solve.**

This final activity could also be the first activity in Step 1. It is placed here because working toward continuously optimizing and refining your strategies, solutions and processes will require an engaged and thriving community of practitioners. A community invested in the continued success and growth of monitoring and observability will have the following qualities:

- **Community-driven:** Optimizing monitoring and observability is more than a set of technologies and methods. It is about gaining the real insight needed to solve problems. Build a healthy community around these solutions to continuously improve the groundwork you've laid as a result of this Solution Path.
- **Leadership-supported:** Community is essential, but it won't flourish without leadership's investment and involvement. Ask leadership to show its support for this community by encouraging participation and making sure those involved have dedicated hours in their work schedule to invest in the community. Make a case to leadership for financial investment in training resources and giving community members opportunities to participate in conferences and other external activities related to monitoring and observability.
- **Grounded in peer-to-peer learning:** Mentor individuals who have become successful practitioners of the solutions and methods described in this Solution Path to become champions within this community, sharing both their successes and failures. Encourage those struggling to share their challenges. Inspire the community by helping those struggling become successful.

Additional Guidance

The purpose of Step 4 is to remind you that this journey is never-ending. Never be discouraged. Discovering a new gap or need is an opportunity to grow and improve. Invest in your collaborative community, be open to new ideas and champion optimizing monitoring and observability in your organization.

Read More

ITSM Best Practices: Implement a Major Incident Management Response Plan

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

ITSM Best Practices: Automating Incident Management

Reference Architecture Brief: Software Observability

Evidence

[1] OODA Loop: Summary of the OODA Model, Value Based Management.net.

[2] What Is OpenTelemetry?, OpenTelemetry.

Document Revision History

Solution Path for Modern Infrastructure and Application Monitoring - 30 November 2020

Recommended by the Author

Some documents may not be available as part of your current Gartner subscription.

Assessing OpenTelemetry's Importance to Application Performance Monitoring

Best Practices for Multicloud Infrastructure Monitoring

Enhance Monitoring and Observability via Telemetry Pipelines

Guidance Framework for Deploying Centralized Log Monitoring

How to Monitor Containers and Kubernetes Workloads

Infographic: Using AIOps to Manage Operational Telemetry

ITSM Best Practices: Automating Incident Management

ITSM Best Practices: Implement a Major Incident Management Response Plan

Monitoring and Observability for Infrastructure and Applications

Reference Architecture Brief: Software Observability

Solution Criteria for AIOps Platforms

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

© 2025 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner is a registered trademark of Gartner, Inc. and its affiliates. This publication may not be reproduced or distributed in any form without Gartner's prior written permission. It consists of the opinions of Gartner's research organization, which should not be construed as statements of fact. While the information contained in this publication has been obtained from sources believed to be reliable, Gartner disclaims all warranties as to the accuracy, completeness or adequacy of such information. Although Gartner research may address legal and financial issues, Gartner does not provide legal or investment advice and its research should not be construed or used as such. Your access and use of this publication are governed by Gartner's Usage Policy. Gartner prides itself on its reputation for independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "Guiding Principles on Independence and Objectivity." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 1: Terminology

Term	Definition
Instrumentation	Code or tooling that captures data about the state of a monitored entity
MTTD	Mean time to detect
MTTR	Mean time to restore
SLI	Service-level indicator: A quantitative measure of some aspect of the level of service being provided
SLO	Service-level objective: A target value or range of values for a service level that an SLI measures
SRE	Site reliability engineering
Telemetry	Data generated by a monitored entity that documents its state

Source: Gartner

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 2: SLI Examples

SLI	Type
The proportion of billing validation requests served in “X” milliseconds or less, measured at the API gateway	Latency
The proportion of valid balance transfer requests served successfully	Availability

Source: Gartner

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 3: Availability Objectives Translated to Time

Availability objective	Outage allowed per year	Outage allowed per quarter	Outage allowed per month	Outage allowed per week	Outage allowed per day	Outage allowed per hour
90%	36.5 days	9 days	3 days	16.8 hours	2.4 hours	6 minutes
95%	18.25 days	4.5 days	1.5 days	8.4 hours	1.2 hours	3 minutes
99%	3.65 days	21.6 hours	7.2 hours	1.68 hours	14.4 minutes	36 seconds
99.5%	1.83 days	10.8 hours	3.6 hours	50.4 minutes	7.2 minutes	18 seconds
99.9%	8.76 hours	2.16 hours	43.2 minutes	10.1 minutes	1.44 minutes	3.6 seconds
99.95%	4.38 hours	1.08 hours	21.6 minutes	5.04 minutes	43.2 seconds	1.8 seconds
99.99%	52.6 minutes	12.96 minutes	4.32 minutes	60.5 seconds	8.64 seconds	9.36 seconds
99.999%	5.26 minutes	1.3 minutes	25.9 seconds	6.05 seconds	0.87 seconds	0.04 seconds

Source: Gartner

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 4: DEM Technologies and Most Common I&O Use Cases

DEM technology	Use cases
Real user monitoring	■ Monitoring performance and quality of user experience for external-facing applications (browser and native mobile app) ■ SLA compliance ■ Root cause analysis of front-end application performance problems ■ User experience analysis through session replay ■ Omnichannel user journey and customer experience analysis ■ Business outcome analysis
Synthetic transaction monitoring	■ Monitoring SaaS applications by simulating multiple step transactions and monitoring the performance at each step ■ Identifying problems across endpoint, local network, internet and application servers ■ Testing from remote locations such as last-mile, cloud providers or Tier 1 providers ■ Benchmarking website availability and performance ■ API monitoring ■ Monitoring wireless access points ■ Monitoring network and internet user traffic

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

- Testing performance in preproduction

Market Guide for Digital Experience Monitoring

Source: Gartner

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 5: Example DEM Vendors With RUM or STM Solutions

Vendor	Real user monitoring	Synthetic transaction monitoring
Apica	NA	Apica Synthetic Monitoring
Broadcom	DX App Experience Analytics	DX App Synthetic Monitor
Catchpoint	Real User Monitoring (RUM)	Synthetic Monitoring
Cisco (AppDynamics)	Browser Real User Monitoring (Browser RUM) Mobile Real User Monitoring (Mobile RUM)	Browser Synthetic Monitoring
Cisco (ThousandEyes)	End User Monitoring	Browser Synthetics
ControlUp	NA	ControlUp Scoutbees
Datadog	Real User Monitoring (RUM)	Synthetic Testing and Monitoring
Dynatrace	Real User Monitoring (RUM)	Synthetic Monitoring
eG Innovations	Real User Monitoring	Synthetic Monitoring
Fortinet	NA	FortiMonitor
ITRS Group	ITRS Uptrends	ITRS Uptrends
Kentik	NA	Synthetic Testing and Monitoring
ManageEngine	Real User Monitoring (RUM), Site24x7	Site24x7
Netskope	Proactive Digital Experience Management (P-DEM)	NA
New Relic	Browser Monitoring	Synthetic Monitoring

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Palo Alto Networks	Autonomous DEM	Autonomous DEM
Riverbed	Aternity Digital Experience Management Platform	Aternity Digital Experience Management Platform
SolarWinds	Pingdom	Pingdom Web Performance Monitor
Splunk	Splunk Real User Monitoring (RUM)	Splunk Synthetic Monitoring
2 Steps	NA	Synthetic Monitoring
Zscaler	NA	Zscaler Digital Experience (ZDX)
NA = Not available		

Source: Gartner

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.

Table 6: Service Topology Technology Examples

Vendors	Solution type	Technology
BMC	Event intelligence	Service Mapping
BigPanda	Event intelligence	Real-time Topology Mesh
Cisco (AppDynamics)	Telemetry analysis	Flow maps
Datadog	Telemetry analysis	Service Map
Dynatrace	Telemetry analysis	Smartscape
Elastic	Telemetry analysis	Service Map
Grafana Labs	Telemetry analysis	Application Observability service map
Honeycomb	Telemetry analysis	Service Map
LogicMonitor	Telemetry analysis	Application Topology Map
Logz.io	Telemetry analysis	Service Map
New Relic	Telemetry analysis	Observability Maps

Source: Gartner

This research note is restricted to the personal use of GERMAN.AGUNDEZ@IKUSI.COM.